RÉCAP · JOURS 1 ET 2

Context Engineering, skills & workflows

Du prompt improvisé au harness maîtrisé

5 Degrés · jeudi 15 et vendredi 16 octobre 2026 · en présentiel

Animateur : Anthony Denneulin

Version de référence : Claude Code 2.1.296 (9 octobre 2026). L'outil change chaque semaine : en cas de doute, la documentation officielle fait foi.

Mode d'emploi

1. Le livret a deux parties : votre cahier, rempli pendant la formation, et ce récap, à relire après : ce qu'il faut retenir de chaque demi-journée, puis quinze lectures pour aller plus loin et les sources.
2. Quatre demi-journées : J1 matin « Comprendre & configurer », J1 après-midi « Hooks & skills », J2 matin « Composer & piloter la boucle », J2 après-midi « Scaler par la parallélisation ».
3. Les commandes, les réglages et les prompts sont reproduits en entier : copiez-les depuis le fichier Markdown ou la version HTML plutôt que de les retaper.
4. Un objectif à 1 ou 2 dans votre auto-évaluation : la dernière colonne du tableau, dans le cahier, indique la partie du récap à reprendre.

Sommaire

Partie	Contenu
J1 matin — Récap	Harness, de l'assistant à l'agent, paysage en quatre étages, chiffres, le goulot déplacé, modes, du prompt au contexte, pilotage du contexte, règle des 70 %, modes de défaillance, prompt précis, CLAUDE.md et votre code
J1 après-midi — Récap	Hooks, protection des secrets, skills et leur partage, qui fait quoi, plugins
J2 matin — Récap	Découvrir un projet avec Claude, le ticket du jour, composer plutôt qu'adopter, la spec, la chaîne de Matt Pocock, doser, superviser, les loops et `/goal`
J2 après-midi — Récap	Le coût de la parallélisation, worktrees, sous-agents, un modèle par phase, fan-out et pipeline, ce qui casse, découper, le graphe des tickets, analyse parallèle, l'Atelier 3 geste par geste, le MCP, mesurer
Pour aller plus loin	Quinze lectures, rangées par demi-journée
Sources des deux jours	Études, articles, exemples, documentation

J1 matin — Récap

Comprendre & configurer. L'agent n'est bon que si ce qu'il voit, son contexte, est bon. Ce matin : le premier contact avec l'agent, la fenêtre de contexte, le prompt précis et le CLAUDE.md.

Le **harness**, c'est tout ce que vous configurez autour du modèle : le contexte (CLAUDE.md, règles, mémoire), les automatismes (hooks), les capacités (skills, sous-agents, MCP) et les barrières (permissions, sandbox). Kief Morris le décrit comme l'ensemble des specs, contrôles qualité et consignes de workflow qui pilotent les agents. Améliorer le harness plutôt que corriger chaque sortie une à une : c'est la position qu'il recommande, « on the loop ».

De l'assistant à l'agent

Étape	Ce que fait l'outil	Votre rôle
Assistant	Complète une ligne ou répond dans un chat ; il ne voit que ce que vous lui montrez	Vous enchaînez : copier, compiler, tester, corriger
Agent	Lit lui-même le repo, modifie plusieurs fichiers, lance des commandes (build, tests, git) et regarde le résultat	Vous définissez la tâche, le contexte et ce que « fini » veut dire
Orchestrateur	Plusieurs agents en parallèle, chacun sur un morceau de la tâche (J2 après-midi)	Vous pilotez, ou un agent principal pilote

Ce qui change d'une étape à l'autre : ce que l'outil peut faire seul, et donc ce qu'il doit savoir. Entre l'assistant et l'agent, ce n'est pas le modèle qui change, c'est l'accès : s'il a mal compris votre projet, l'agent se trompe à grande échelle, sur dix fichiers au lieu d'une ligne.

Un agent = un modèle + des outils + une loop. Il lit (des fichiers, le code, la documentation du projet), agit (il édite, il exécute des commandes), vérifie (les tests, le lint), et chaque résultat redevient du contexte pour le tour suivant, jusqu'à ce qu'il juge la tâche finie : c'est la loop la plus simple, turn-based. Il ne vérifie que si on lui donne de quoi vérifier : sur un projet sans tests, sans lint, sans commande de build, vérifier se réduit à relire son propre code. Le modèle, vous ne le changez pas ; les outils et la loop, vous les configurez : c'est le harness.

Le paysage en quatre étages

Étage	Ce qu'il fait	Exemples
Surfaces de pilotage	Là où vous lancez, suivez et validez	Terminal, IDE, app desktop, web ; ADE (environnements de développement agentiques) comme Conductor ou Orca, qui font tourner plusieurs agents en parallèle
Votre harness	Ce que vous configurez par projet : ces deux jours	CLAUDE.md, règles, mémoire, hooks, skills, sous-agents, MCP, permissions
Agents de code	La loop, les outils, les permissions	Claude Code, Codex, Copilot, Cursor ; ouverts : Pi (minimal), OpenCode
Modèles	Génèrent le texte et les appels d'outils	Claude, GPT, Gemini ; open-weight : DeepSeek, Kimi, GLM

On fait un choix à chaque étage, et les étages se branchent les uns sur les autres : un même modèle tourne dans plusieurs agents. Votre harness vous suit en grande partie d'un agent à l'autre, grâce aux standards ouverts (AGENTS.md, Agent Skills, MCP). Le vibe coding (Lovable, Bolt, v0, Replit) fusionne les quatre étages

dans un seul produit web : rapide pour un prototype, avec une prise limitée sur votre harness : des consignes de projet, parfois des skills, dans le cadre que fixe le produit. Schéma d'après l'« Agent Harness Map » de Florian Bruniaux.

Adoption et confiance en 2026

Chiffre	Ce qu'il mesure	Source
90 %	des développeurs professionnels utilisent des agents de code au moins chaque semaine ; 68 % chaque jour. Un an plus tôt, la même enquête mesurait encore l'usage de l'IA pour coder en général : 85 % en 2025	JetBrains, résultats partiels mai-juillet 2026 ; 2025, 24 534 développeurs
48 %	ne font confiance à l'IA que quand la sortie est facile à vérifier	Stack Overflow 2026, environ 31 000 réponses
63 %, 61 % et 20 %	citent l'information incomplète, le contexte resté dans la tête des gens, et des outils IA qui manquent de contexte, parmi ce qui les prive du bon contexte au travail	Stack Overflow 2026
+19 %	de temps sur les tâches avec l'IA, alors que les développeurs pensaient en avoir gagné 20 %	METR, juillet 2025, 16 développeurs open source expérimentés

La confiance vient de la vérification et du contexte. En tête de ce qui prive du bon contexte, des obstacles humains ; les outils IA n'arrivent qu'en 9e position : le contexte qui manque à l'agent manque d'abord aux humains, et l'écrire, dans un CLAUDE.md par exemple, sert les deux. Sur METR : il juge aujourd'hui ses résultats de 2025 dépassés ; une mise à jour de février 2026 donne des chiffres plus favorables, mais METR les juge lui-même peu fiables et change de protocole.

Le goulot s'est déplacé

Avec un agent, l'écriture du code (le « build ») s'effondre à la vitesse de l'agent ; les étapes autour tournent toujours à vitesse humaine (Anthropic, « The AI-native SDLC playbook », 21/08/2026, figure 1 : « The bottleneck moved »). Ce qui reste lent : dire quoi construire, et vérifier. C'est le plan des deux jours. En amont, dire quoi construire : le contexte, le prompt précis et le CLAUDE.md (J1 matin), la spec et les tickets (J2 matin). En aval, vérifier : les hooks (J1 après-midi), `/goal` (J2 matin), un relecteur en contexte frais (J2 après-midi). Le temps de cycle récupéré (« cycle time reclaimed », en pointillés sur le schéma) n'arrive que si la relecture suit ; sinon, il se perd en file d'attente devant les relectures.

Les modes et les raccourcis

Shift+Tab fait défiler les quatre premiers modes ; le cinquième ne s'obtient qu'au lancement.

Mode	Ce que fait Claude	Quand l'utiliser
Auto	Les actions jugées sûres passent sans vous demander. C'est le mode de départ pour tous depuis fin septembre 2026 (v2.1.283-2.1.284).	Le quotidien, sur un repo bien configuré (permissions, hooks, tests).
Manual	Demande votre accord avant chaque action sensible.	Un repo sensible, ou quand vous voulez tout valider.
Accept edits	Modifie les fichiers sans demander.	Une tâche bien cadrée : plan validé ; vous relirez le diff.
Plan	Explore, puis propose un plan, sans modifier vos fichiers. Pas tout à fait de la lecture seule : quand le mode Auto est disponible, les commandes que son classifieur juge sûres peuvent tourner.	La première fois sur un repo ; une tâche floue ou qui touche plusieurs fichiers.
Bypass permissions	Seulement au lancement, avec `claude --dangerously-skip-permissions`. Tout passe sans demander, même l'écriture dans `.git` et `.claude`, et sans le classifieur du mode Auto ; vos règles `deny` et vos hooks bloquent encore.	Seulement dans un conteneur ou une VM isolés, sans accès à internet. Sur votre poste, pour moins de demandes : le mode Auto.

Raccourci	Effet
Shift+Tab	Passer au mode suivant
Esc	Interrompre Claude, pour le réorienter
Esc Esc	Revenir en arrière (`/rewind`)
`@chemin/fichier`	Référencer un fichier dans le prompt
`/plan <description>`	Entrer directement en Plan mode avec une tâche

Les permissions. Quand Claude demande une permission, les réponses habituelles sont : oui ; oui, et ne plus demander ; ou non (une option propose souvent aussi de passer en mode Auto). Relisez le « ne plus demander » : pour une commande, il enregistre souvent un motif plus large que la commande tapée, comme `npm test *` ; pour une modification de fichier, il ne vaut que jusqu'à la fin de la session. Le projet peut pré-autoriser des commandes dans `.claude/settings.json` (`permissions.allow`) ou en interdire (`permissions.deny`). Bénéfriches, le projet de démo, pré-autorise 45 commandes pnpm précises : `pnpm --filter shared test` passe sans question.

Du prompt au contexte

Avec un chat, l'effort portait sur le message : c'était le prompt engineering. Avec un agent, chaque action produit un résultat (un fichier lu, une sortie de tests) qui redevient du contexte : la fenêtre se remplit toute seule, à chaque tour de loop, et votre prompt n'en est qu'une petite partie. Le context engineering, c'est choisir ce qui entre :

- l'art de remplir la fenêtre de contexte avec exactement l'information utile à l'étape suivante (Andrej Karpathy, juin 2025, paraphrase) ;
- chercher le plus petit ensemble de tokens à forte valeur qui maximise la probabilité du résultat voulu (Anthropic, septembre 2025, paraphrase).

Piloter la fenêtre de contexte

Le modèle relit tout, à chaque tour : le CLAUDE.md, vos messages, ses réponses, les fichiers lus, les sorties de commandes. En fin de session, ce sont les résultats d'outils qui occupent l'essentiel de la fenêtre, pas vos prompts. Et plus de contexte ne donne pas une meilleure réponse : la qualité baisse quand l'entrée s'allonge, même sur des tâches simples (Chroma, « Context Rot », 2025, 18 modèles dont Claude). Le modèle a un budget d'attention limité, et chaque token ajouté en consomme une part (Anthropic, 2025) : un fichier hors sujet ne fait pas que prendre de la place, il dilue l'attention portée à ce qui compte. Une fenêtre de 1M de tokens n'est pas une invitation à la remplir.

L'effet « lost in the middle » (une information placée au milieu d'un long contexte est moins bien exploitée, Liu et al., 2023) est contesté sur les modèles récents. Le conseil qui reste vrai : les documents longs en tête, la question à la fin.

Commande	Ce qu'elle fait	Quand l'utiliser
`/context`	Montre l'occupation de la fenêtre par poste (fichiers mémoire, skills, outils, MCP, messages), avec des suggestions.	Avant une grosse tâche ; quand Claude semble oublier une consigne ; après avoir branché un serveur MCP.
`/compact [consigne]`	Résume la conversation en gardant ce que dit la consigne. Recharge ensuite le CLAUDE.md racine, les règles sans `paths` et la mémoire automatique ; les règles avec `paths` reviennent quand Claude relit un fichier qui correspond. Sans consigne, c'est le modèle qui choisit ce qu'il garde.	Même tâche, contexte lourd. Exemple : `/compact garde la liste des fichiers modifiés et les commandes de test`.
`/clear`	Ouvre une nouvelle conversation, contexte vide. Le CLAUDE.md et la mémoire automatique restent, relus : une modification du CLAUDE.md est prise en compte. `/resume` retrouve l'ancienne conversation.	Tâche terminée ; contexte empoisonné par une erreur qui revient ; après avoir modifié le CLAUDE.md.
`/rewind` (Esc Esc)	Restaure le code et/ou la conversation à un checkpoint (un avant chaque prompt). Ne suit pas les modifications faites par Bash ni par un sous-agent. Ne remplace pas git.	Claude part dans une mauvaise direction.
`/autocompact <taille>`	Règle le seuil de l'auto-compaction ; `auto` revient au réglage par défaut.	Une fois, pour appliquer la règle des 70 %.
`/memory`	Ouvre les fichiers mémoire (CLAUDE.md…) ; active ou désactive la mémoire automatique.	Vérifier ce qui est chargé ; corriger une consigne.
`/init`	Génère un CLAUDE.md de départ à partir du code.	Repo sans CLAUDE.md ni AGENTS.md : un point de départ, à relire et à élaguer. Pas sur un repo qui n'a qu'un AGENTS.md : dès qu'un CLAUDE.md existe, Claude Code cesse de lire l'AGENTS.md, sauf à l'importer avec `@AGENTS.md`.

La règle des 70 %

C'est une heuristique, pas une fonction de Claude Code : compactez vers 60 % de la fenêtre, ne dépassez pas 70 %. Par défaut, l'auto-compaction se déclenche près de la limite (environ 967K tokens sur une fenêtre de 1M). Un second déclencheur, annoncé avec la v2.1.299 et activé à distance, tourne aussi sur des versions plus anciennes : sur un abonnement, en session interactive, une conversation de plus de 200K tokens restée inactive environ 54 minutes, sur un ordinateur qui n'est pas en veille, peut être compactée d'office.

`"idleCompaction": false` le coupe à partir de la v2.1.290 ; avant, il faut couper toute l'auto-compaction. Pour appliquer la règle, réglez le seuil à 70 % de la taille de votre fenêtre ; depuis la v2.1.288, le réglage est mémorisé pour le modèle utilisé.

```
/autocompact 700k
```

Sur une fenêtre de 1M, cela déclenche l'auto-compaction vers 70 %. Adaptez le nombre à la taille de votre fenêtre, que `/context` affiche. `/autocompact auto` revient au réglage par défaut.

Situation	Commande
Tâche terminée	`/clear`
Même tâche, contexte lourd	`/compact <ce qu'il faut garder>`
Mauvaise direction	Esc Esc (`/rewind`)
Savoir où on en est	`/context`

Les 4 modes de défaillance

D'après Drew Breunig (2025), un contexte long dégrade les réponses de quatre façons. La colonne « Parade » traduit chaque mode en gestes Claude Code.

Mode	Ce qui se passe	Exemple de dev	Parade
Poisoning (empoisonnement)	Une erreur entre dans le contexte et y est réutilisée.	Claude invente une méthode d'API, puis la réutilise dans toute la session.	Esc Esc pour revenir avant l'erreur, ou `/clear` et un prompt qui donne la bonne information.
Distraction	Le contexte est si long que le modèle s'y accroche et néglige ce qu'il a appris.	Après trois heures de session, Claude répète ses actions passées.	`/compact` avec une consigne, ou `/clear` et repartir d'un résumé.
Confusion	Du contenu superflu, par exemple trop d'outils, dégrade la réponse.	Trente outils MCP chargés pour une tâche qui en utilise deux.	`/context` pour repérer les postes inutiles ; débrancher les serveurs MCP dont la tâche n'a pas besoin.
Clash (conflit)	Des informations ajoutées contredisent ce qui est déjà là.	Le CLAUDE.md dit Jest, un message plus loin dit Vitest.	Corriger la source (le CLAUDE.md) plutôt que contredire dans la conversation.

Un prompt précis : quatre verbes

Verbe	Ce que vous donnez	Exemple
Référencer	Un chemin, un symbole, avec `@`	`@src/app/orders/order-list.component.ts`
Montrer	Un exemple, un fichier modèle	« sur le modèle de `order-list.component.ts` »
Contraindre	Ce qu'il ne faut pas toucher, le critère de vérification	« Ne touche pas à `ProductService`. »
Structurer	Un plan d'abord, des étapes, un format de sortie	« Propose-moi d'abord un plan. »

Exemple Angular

Avant : « Ajoute un filtre à la liste des produits. »

Après :

> Dans `@src/app/products/product-list.component.ts`, ajoute un filtre par catégorie sur le modèle de `order-list.component.ts` (même pattern signal + computed). Ne touche pas à `ProductService`. Ajoute un test dans `product-list.component.spec.ts` ; `npm test -- product-list` doit passer. Propose-moi d'abord un plan.

Les quatre verbes : référencer (le chemin avec `@`), montrer (`order-list.component.ts`), contraindre (`ProductService` intouché, le test qui doit passer), structurer (un plan d'abord).

Exemple .NET

Avant : « Corrige le bug des commandes. »

Après :

> `OrdersController.Create` (`src/Api/Controllers/OrdersController.cs`) renvoie 500 quand `ShippingAddress` est null. Écris d'abord un test xUnit qui reproduit le bug dans `OrdersControllerTests`, puis corrige. `dotnet test --filter OrdersController` doit passer.

Contraindre, c'est donner un critère de vérification

Sans critère, l'agent s'arrête quand il pense avoir fini. Avec un critère, il vérifie lui-même, et recommence si ça échoue. Les meilleurs critères sont rapides, ciblés et déterministes : un test unitaire sur un fichier, le lint, le compilateur ; pas une suite e2e de vingt minutes à chaque itération. Pas de tests sur la zone ? Demandez-lui d'écrire le test d'abord, puis relisez-le vous-même avant de le laisser implémenter : vous relisez un test de quinze lignes plutôt qu'un diff de deux cents, et le test devient votre spécification. C'est le fil rouge des deux jours : un hook Stop cet après-midi, `/goal` demain.

CLAUDE.md, le README des agents

Claude Code charge `CLAUDE.md` à chaque session. Depuis la v2.1.277, il ne lit `AGENTS.md` que s'il ne trouve ni `CLAUDE.md` ni `CLAUDE.local.md` dans le dossier ou au-dessus ; si les deux existent, seul le `CLAUDE.md` compte, et la ligne `@AGENTS.md` dans le `CLAUDE.md` permet de partager un seul fichier entre plusieurs outils. C'est un conseil, pas une garantie : ce qui doit arriver à chaque fois se branche dans un hook (cet après-midi).

Les 6 zones

Zone	La question	Exemple Angular / .NET
Commands	Quelles commandes un nouveau ne devinerait pas ?	`npm run test:ci` ; `dotnet test --filter Category=Unit`
Testing	Quel framework, quelles conventions ?	Jest et Testing Library ; xUnit et FluentAssertions
Pièges & environnement	Qu'est-ce qui surprend un nouveau ?	L'API doit tourner avant `ng serve` ; `dotnet ef` exige `--startup-project`
Code style	Quels patterns propres au projet ?	« Suis `src/app/orders/` »
Git workflow	Qu'est-ce qui n'est pas standard ?	Conventional commits, squash
Boundaries	Que ne touche-t-on jamais ?	`appsettings.Production.json`, `.env*`, migrations déjà appliquées

Le vrai exemple de Bénéfriches : son `AGENTS.md` (87 lignes) donne l'identité du projet en deux paragraphes, les workspaces (chacun a son propre AGENTS.md), l'index des règles, les conventions, un tableau « quel test lancer selon ce qui change » et la politique git.

Ce qu'on n'y met pas

À éviter	À faire
Ce que Claude déduit du code	Ce que Claude ne peut pas deviner
La description du code fichier par fichier	Les pièges et l'environnement : ce qui surprend un nouveau
Les consignes vagues (« écris du code propre »)	Un fichier modèle, des règles, un hook de lint
Un fichier qui grossit sans fin	Moins de 200 lignes

Du code propre, ça se spécifie : un fichier modèle, des règles, un hook de lint.

Et formulez des principes plutôt que des règles rigides. Dans le system prompt de Claude Code, Anthropic a remplacé « par défaut, n'écris aucun commentaire ; jamais de docstring de plusieurs paragraphes » par « écris du code qui se lit comme le code qui l'entoure : même densité de commentaires, même nommage, mêmes idiomes » (« The new rules of context engineering for Claude 5 generation models », 24/07/2026). La règle impose la même valeur partout ; le principe laisse Claude juger avec le code sous les yeux.

Votre code est le premier contexte

Une étude d'ETH Zurich (Gloaguen et al., version de septembre 2026) a fait tourner Claude Code, Codex et Qwen Code sur des tâches réelles, avec et sans fichier de contexte : pas de meilleur taux de réussite en général, et plus de 20 % de coût d'inférence en moyenne. Les consignes explicites sont bien suivies, utiles pour les pratiques non standard ; les vues d'ensemble du dépôt n'aident pas, parce qu'elles répètent la documentation déjà présente. Ce que Claude lit pour comprendre votre code, c'est le code :

Ce que Claude lit	Chez vous
Des noms explicites	`calculateShippingFee()` plutôt que `calc()`
Une structure prévisible	Une feature Angular par dossier, des controllers .NET à leur place
Des types stricts	TypeScript en mode `strict`, `<Nullable>enable</Nullable>` en C#
Des tests	Des exemples d'usage, et une commande qui vérifie

Code lisible : CLAUDE.md court. Code confus : le CLAUDE.md doit tout compenser, et ce qui compense vieillit.

Le golden file

Plutôt que d'écrire vos règles de style à la main, désignez un fichier modèle. Deux lignes suffisent :

```
## Code style
- Angular : suis `src/app/orders/` pour toute nouvelle feature.
- .NET : suis `OrdersController` pour tout nouveau controller.
```

Bénéfriches commence ainsi sa règle sur les controllers : « Reference controller: sites.controller.ts ». La suite précise ce que le modèle montre mal : la forme des réponses, la validation, le mapping des erreurs, et les anciennes routes de ce fichier, à ne pas copier. Zéro maintenance pour tout le reste : Claude lit le modèle quand il en a besoin.

Une condition : le fichier modèle doit être exemplaire. Choisissez une feature récente, propre et testée ; si vous pointez du code legacy, Claude en copiera fidèlement les défauts.

La hiérarchie

Fichier	Portée	Chargé
`~/.claude/CLAUDE.md`	Vous, dans tous vos projets	Au démarrage
`./CLAUDE.md` (ou `AGENTS.md` s'il n'y a ni `CLAUDE.md` ni `CLAUDE.local.md`)	Le projet, versionné : toute l'équipe	Au démarrage
`CLAUDE.md` d'un sous-dossier	Une partie du repo	Quand Claude travaille dans ce dossier
`.claude/rules/*.md` sans `paths`	Le projet	Au démarrage
`.claude/rules/*.md` avec `paths`	Les fichiers qui correspondent aux globs	Quand Claude lit ou modifie un fichier correspondant
`~/.claude/projects/<projet>/memory/MEMORY.md`	La mémoire automatique, écrite par Claude : cet index, et une note par fichier	L'index au démarrage (les 200 premières lignes ou 25 Ko) ; les notes à la demande

La mémoire automatique ne remplace pas le CLAUDE.md : elle n'est pas versionnée, donc pas partagée avec l'équipe. Mais elle sert de révélateur : ouvrez-la de temps en temps avec `/memory`. Ce que Claude y range, surtout les corrections que vous lui avez données (« utilise pnpm, pas npm »), manque souvent au CLAUDE.md : promouvez-le dans le fichier versionné, pour toute l'équipe.

Une règle avec `paths`

Fichier `.claude/rules/angular.md` :

```
---
paths:
  - "src/app/**/*.ts"
---

# Angular

- Nouvelle feature : suis `src/app/orders/`.
- Le test vit à côté du composant : `*.component.spec.ts`.
```

Cette règle ne coûte rien tant que Claude ne touche pas à un fichier `src/app/**/*.ts`. Bénéfriches a six règles de ce type (tests, controllers, migrations…). Pour vérifier qu'une règle se charge : faites lire à Claude un fichier qui correspond, et guettez la ligne « Loaded .claude/rules/… » qui s'affiche alors dans le terminal. `/context` montre surtout ce qui a été chargé au lancement.

L'entretenir

Trois commandes intégrées à Claude Code, rien à installer :

- `/doctor prompt-audit` (version 2.1.283 ou plus) audite vos fichiers d'instructions (CLAUDE.md, CLAUDE.local.md, AGENTS.md, règles, skills, sous-agents) : instructions écrites pour d'anciens modèles, références à des fichiers ou commandes qui n'existent plus, fichiers qui se contredisent. Il rend un rapport et ne modifie rien sans votre accord. `/doctor prompt-audit .claude/rules` limite l'audit à un dossier.
- `/doctor` fait le bilan complet : pour le CLAUDE.md, il propose de couper ce que Claude peut déduire du code et de déplacer ce qui ne sert que parfois.
- `/insights` analyse vos sessions récentes sur cette machine (jusqu'à 200 à la fois) et écrit un rapport HTML dans `~/.claude/usage-data/report.html` : vos projets, votre usage, ce qui frotte (demandes mal comprises, code bogué) et des fonctions à essayer. C'est là qu'on repère l'erreur qui revient. Les sessions sont locales, mais l'analyse passe par le modèle et consomme votre forfait.

Le test de la documentation officielle, pour chaque ligne : « si je la retire, Claude fera-t-il une erreur ? » Si non, coupez. Et chaque erreur qui revient devient une règle.

J1 après-midi — Récap

Hooks & skills. CLAUDE.md décrit, les hooks garantissent, les skills complètent à la demande. Cet après-midi : les automatismes, les barrières et les capacités du harness.

Les hooks : du conseil à la garantie

	CLAUDE.md (conseil)	Hook (garantie)
Ce que vous écrivez	« Formate avec Prettier »	Un script lancé à chaque modification
Ce qui se passe	Suivi la plupart du temps	Exécuté à chaque fois

Ce qui doit arriver à chaque fois ne s'écrit pas dans le CLAUDE.md : ça se branche dans un hook.

Anatomie d'un hook

Élément	Rôle	Exemple
Événement	Le moment où le hook se déclenche	`PreToolUse` : avant chaque appel d'outil
Matcher	Filtre sur le nom de l'outil	`Bash` ; ou `Edit` et `Write`
Handler	Ce qui est lancé, le plus souvent une commande. Elle reçoit un JSON sur stdin.	`node "${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-env.mjs"`
Code de sortie	La réponse du hook	0 : on continue ; 2 : on bloque

Ce que reçoit le script sur stdin (extrait) :

```
{
  "hook_event_name": "PreToolUse",
  "cwd": "/home/vous/mon-projet",
  "tool_name": "Bash",
  "tool_input": {
    "command": "cat .env"
  }
}
```

Pour Read, Edit et Write, le chemin du fichier arrive dans `tool_input.file_path` (chemin absolu).

Le matcher compare des noms d'outils exacts, majuscule comprise : `Bash`, pas `bash`. Dans `Edit|Write|Bash`, la barre verticale veut dire « ou » ; avec d'autres caractères (`.*`, `^`…), le matcher devient une expression régulière, et `Edit` attraperait aussi `NotebookEdit`.

Les événements utiles

Claude Code en compte 33. Six suffisent pour commencer.

Événement	Quand	Exemple
`SessionStart`	Au démarrage de la session ; avec le matcher `compact`, après chaque compaction	Recharger le contexte
`UserPromptSubmit`	À chaque prompt	Ajouter une information
`PreToolUse`	Avant un outil ; peut bloquer	Protéger un fichier
`PostToolUse`	Après un outil ; ne peut plus bloquer	Formater le fichier modifié
`Stop`	Quand Claude veut s'arrêter	Lancer les tests
`Notification`	Quand Claude attend votre permission ou votre réponse	Vous prévenir

Les codes de sortie

Code	Effet
0	Tout va bien, l'action continue.
2	Bloque, et le message écrit sur stderr est renvoyé à Claude. `PreToolUse` : l'appel est annulé. `PostToolUse` : rien à annuler, l'outil a déjà tourné. `Stop` : Claude ne s'arrête pas.
Autre (1…)	Erreur non bloquante : l'action continue. Attention, `exit 1` ne bloque rien. Un hook qui plante laisse donc passer, sauf avec `"onFailure": "block"` (v2.1.295), et sauf, depuis la v2.1.299, un hook `PreToolUse` sur un poste Windows sans Git Bash : l'appel est alors bloqué. Un hook de protection sort explicitement en 2, et se teste à la main avant qu'on lui fasse confiance.

La configuration

Les hooks se déclarent dans `.claude/settings.json` (versionné : toute l'équipe) ou dans `.claude/settings.local.json` (personnel). La structure : `hooks` → événement → liste de groupes `{ matcher, hooks }` → handlers `{ type, command }`. Voici l'exemple officiel, qui formate chaque fichier modifié (jq requis) :

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
          }
        ]
      }
    ]
  }
}
```

- Le chemin du fichier arrive dans le JSON sur stdin : il n'existe pas de variable `$FILE_PATH`.
- `${CLAUDE_PROJECT_DIR}` donne la racine du projet : appelez toujours un script de `.claude/hooks/` par ce chemin, jamais par un chemin relatif. Les hooks tournent dans le dossier courant de Claude : après un `cd`, le script est introuvable, et le hook ne protège plus rien (sous Windows sans Git Bash, il bloque chaque action) ; seule une « hook error » à chaque appel le signale. Gardez les accolades : sous Windows sans Git Bash, les hooks tournent sous PowerShell, et Claude Code ne réécrit pour lui que cette forme.

- jq n'est pas toujours installé, sous Windows notamment : les scripts des exercices sont en Node.js.
- `/hooks` liste les hooks chargés, avec le fichier d'où vient chacun et la commande complète. Il sert à regarder : on corrige dans le fichier de settings.
- `claude --debug=hooks` (avec le signe égal) écrit un log des hooks dans `~/.claude/debug/`, un fichier par session, avec une ligne par hook lancé et son code de sortie. Rien ne s'affiche dans le terminal.
- Un hook de type commande s'exécute avec tous vos droits d'utilisateur. Les hooks du `.claude/settings.json` d'un repo cloné attendent que vous acceptiez la confiance du dossier : relisez son `.claude/` avant. Avec `claude -p`, il n'y a pas de dialogue : ils tournent sans rien demander. Avant de scripter `claude -p` sur un repo que vous n'avez pas écrit, relisez ses settings, ou coupez ses hooks pour ce lancement : `--settings '{"disableAllHooks": true}'`.

Deux hooks de confort

Être prévenu quand Claude vous attend, dans vos settings personnels (`~/.claude/settings.json`), sur macOS :

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "permission_prompt|idle_prompt",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude Code vous attend\" with title
\"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

Dans le terminal, l'alerte arrive après un délai : environ 6 secondes pour une demande de permission, 60 secondes après la fin d'une réponse. Si rien ne s'affiche, lancez une fois `osascript -e 'display notification "test"'`, puis autorisez l'Éditeur de scripts dans Réglages Système, Notifications : sans cette autorisation, la commande échoue sans rien dire.

Réinjecter l'essentiel après chaque compaction, dans `.claude/settings.json` : ce que la commande écrit sur stdout est ajouté au contexte de Claude.

```
{
  "hooks": {
    "SessionStart": [
      {
        "matcher": "compact",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Rappel : pnpm, pas npm.'; git log --oneline -5"
          }
        ]
      }
    ]
  }
}
```

Pour ce qui sert à chaque session, c'est le CLAUDE.md, pas un hook.

Un hook Stop qui lance les tests

Quand Claude veut s'arrêter, le hook lance vos tests. S'ils sont rouges, il sort en code 2 avec la fin de la sortie : Claude ne s'arrête pas et corrige. Le script `tests-verts.mjs` est dans le cahier, en bonus de l'exercice `.env`.

- Le JSON du hook Stop contient `stop_hook_active`, vrai quand Claude continue déjà à cause d'un hook Stop. Le script doit le tester, pour ne pas tourner en rond.
- Il tourne à chaque fin de tour : réservez-le à des tests rapides.
- Claude Code passe outre un hook Stop qui bloque huit fois de suite sans que Claude ait appelé d'outil entre-temps. Le compteur repart à zéro à chaque appel d'outil : ce plafond ne remplace pas le test de `stop_hook_active`.

Hooks Claude Code et hooks git

	Hooks Claude Code	Hooks git
Quand	Pendant la session, sur chaque action de l'agent	Au commit ou au push
Pour qui	L'agent	Tout le monde, humain ou agent
Exemples	Prettier après chaque modification ; protection du `.env`	Husky, lint-staged, Talisman (détection de secrets)

Le pont, chez Bénéfriches : le hook `block-hook-bypass.mjs` empêche Claude de lancer `git commit --no-verify` et ses variantes. Le pre-commit (Talisman, lint) tourne donc même si Claude tente les contournements courants. Pour mettre en place Husky, lint-staged, le typecheck et les tests au commit : le skill `setup-pre-commit` de Matt Pocock, hors de son plugin (dossier `skills/misc/` de son dépôt), à copier dans `.claude/skills/`.

Protéger un secret : quatre niveaux

Niveau	Ce que c'est	Ce qu'il couvre	Ce qu'il ne couvre pas
1. CLAUDE.md	Une consigne : « ne lis jamais le `.env` »	Rien de garanti	Tout : c'est un conseil
2. `permissions.deny`	Des règles déclaratives : `"Read(**/.env)"`, `"Edit(**/.env)"`	Read, Edit et Write sur ce chemin ; les commandes de fichiers que Claude Code reconnaît (`cat`, `head`, `grep` sur le fichier nommé)	Un script qui ouvre le fichier lui-même (`node -e`), un joker du shell (`cat .en*`)
3. Hook `PreToolUse`	Un script qui inspecte chaque appel	Les outils de fichiers, et le texte des commandes	Une commande assez tordue (nom de fichier fabriqué, joker du shell)
4. Sandbox	Une isolation par le système, à activer	Les commandes shell et leurs sous-processus, quel que soit leur texte ; il reprend vos règles `Read` deny	Les outils de fichiers, les serveurs MCP, les hooks ; Windows natif (macOS, Linux et WSL2 seulement) ; la relance hors du sandbox d'une commande qui y a échoué, à fermer avec `"sandbox": { "allowUnsandboxedCommands": false }`

Chaque niveau couvre les trous du précédent. Deux précisions : un hook qui renvoie `allow` ne lève jamais un `deny` ; un fichier `.claudeignore` n'a aucun effet. Le sandbox est désactivé par défaut et s'active avec `/sandbox` ; la documentation prévient qu'il réduit le risque sans être une isolation complète.

Les skills : du savoir ou des procédures, à la demande

Quand créer un skill

D'après la documentation d'Anthropic : quand vous recollez sans cesse le même playbook ou la même procédure dans le chat, ou quand une section du CLAUDE.md est devenue une procédure plutôt qu'un fait. Deux exemples : créer une migration de base de données selon vos conventions ; faire une revue de code selon votre checklist. Un skill peut aussi porter un savoir qui ne sert que pour une tâche, comme le fonctionnement d'un ancien module : le critère, c'est la fréquence d'usage, pas la nature du contenu.

	CLAUDE.md	Skill
Chargement	Toujours : coût permanent	À la demande
Contenu	Ce qui sert à chaque session : faits, conventions, commandes	Ce qui sert à une tâche : un savoir ou une procédure
Portée	Tout le projet	Une tâche

Toujours pertinent : CLAUDE.md. Parfois pertinent : skill.

Structure et emplacements

Un skill est un dossier qui contient un `SKILL.md` (frontmatter et instructions) et, au besoin, des fichiers annexes.

```
.claude/skills/
└── creer-migration/
    ├── SKILL.md        frontmatter + instructions
    ├── reference.md    détails, lus à la demande
    ├── examples/       exemples
    └── scripts/        scripts que le skill lance
```

Emplacement	Pour qui
`~/.claude/skills/<nom>/`	Vous, dans tous vos projets
`.claude/skills/<nom>/`	Le projet, versionné : toute l'équipe l'a au prochain pull
Dossier `skills/` d'un plugin	Ceux qui installent le plugin

Bénéfriches range ses 13 skills dans `.agents/skills/` et les relie à `.claude/skills/` par des liens symboliques : Claude et Codex lisent les mêmes fichiers, car les skills suivent un standard ouvert.

Le frontmatter : les champs utiles

Champ	Rôle
`name`	Le nom à taper après `/` (par défaut, celui du dossier)
`description`	Ce que fait le skill et quand l'utiliser. C'est elle qui le déclenche.
`when_to_use`	Phrases de déclenchement, exemples de demandes ; ajoutées à la description
`argument-hint`	Les arguments attendus, affichés dans le menu
`disable-model-invocation`	`true` : seul l'humain l'invoque (`/nom`)
`user-invocable`	`false` : absent du menu `/`, seul Claude l'invoque
`allowed-tools`	Pré-approuve des outils pendant le tour d'invocation. Ne restreint rien.

Champ	Rôle
`disallowed-tools`	Retire des outils pendant l'exécution du skill
`model`, `effort`	Modèle et niveau d'effort (low, medium, high, xhigh, max)
`context: fork` et `agent`	Exécute le skill dans un sous-agent
`paths`	Globs : Claude ne l'active seul que sur les fichiers correspondants

Un exemple réel, le skill `create-database-migration` de Bénéfriches (extrait) :

```
---
name: create-database-migration
description: Create Knex database migrations for the Benefriches API. Use when adding, modifying, or
removing database columns/tables. (…)
effort: low
allowed-tools: Bash, Read, Write
---
```

La description dit ce que fait le skill (« Create Knex database migrations… »), puis quand l'utiliser (« Use when… »).

Son corps décrit la commande `pnpm --filter api knex:new-migration <nom>`, la mise à jour de `tableTypes.d.ts`, et un piège : ne pas ouvrir de transaction dans une migration Knex, elle est déjà transactionnelle.

Le chargement progressif

Moment	Ce qui entre dans le contexte
Au démarrage	Le nom et la description (+ `when_to_use`) de chaque skill que Claude peut invoquer. Budget global d'environ 1 % de la fenêtre ; 1 536 caractères au plus par skill.
À l'invocation	Le `SKILL.md` complet
À la demande	Les fichiers annexes

Au-delà du budget, les descriptions des skills les moins utilisés tombent ; les noms restent. Exemple : le skill `code-reviewer` de Bénéfriches ne lit ses checklists par domaine (`reference/api.md`, `reference/web.md`) que si les fichiers modifiés le demandent. `/skill-doctor` montre ce que coûte chaque skill chargé et lesquels ne servent jamais : de quoi faire le ménage.

Qui déclenche le skill

Configuration	Vous (`/nom`)	Claude, tout seul	Pour quoi
Par défaut	Oui	Oui	Revue de code, création de composant
`disable-model-invocation: true`	Oui	Non	Ce qui a des effets de bord : déployer, committer
`user-invocable: false`	Non	Oui	Des conventions que Claude applique en arrière-plan

Si Claude ne déclenche pas votre skill, c'est presque toujours la description : elle doit dire ce que fait le skill et quand l'utiliser (« À utiliser quand … »), avec les mots que vos collègues emploient vraiment : « migration », « colonne », « table », pas « évolution du schéma de persistance ». Testez-la sur un cas réel, sans nommer le

skill : Claude l'invoque-t-il seul ? Sinon, retravaillez-la. Plus rare, un frontmatter invalide : le skill se charge quand même, mais sans sa description ; `claude plugin validate .claude/skills` montre l'erreur, comme le journal de `claude --debug`.

Écrire un bon skill

- Une seule responsabilité.
- N'écrivez pas l'évident : seulement ce qui sort Claude de ses habitudes, sur ce projet-là.
- Des étapes qui finissent par une vérification : une commande de test ou de lint. Un skill de savoir n'a pas d'étapes : seulement ce qu'il faut savoir, et les pièges.
- Une section « Pièges » (Gotchas), enrichie à chaque usage : ce que Claude rate sur votre projet.
- Un `SKILL.md` court (moins de 500 lignes) ; les détails dans des fichiers annexes.
- Commencez simple : `/grill-me` de Matt Pocock tient en une ligne, qui appelle un skill d'une trentaine de lignes, et produit des sessions de dizaines de questions.

Partager un skill

Niveau	Comment	Pour qui
1. Dans le repo	Versionnez `.claude/skills/` : le skill évolue par pull request, comme le code.	L'équipe du projet, au prochain pull
2. Dans un plugin	Publiez-le dans une marketplace (en pratique, un dépôt git), déclarée et activée dans le `.claude/settings.json` du repo.	Plusieurs repos
3. Au-delà de Claude	Les skills suivent un standard ouvert.	D'autres agents : Bénéfriches partage les siens avec Codex

Le bloc du niveau 2, dans `.claude/settings.json` (les noms sont des exemples) :

```
{
  "extraKnownMarketplaces": {
    "outils-equipe": {
      "source": { "source": "github", "repo": "votre-org/claude-plugins" }
    }
  },
  "enabledPlugins": {
    "conventions-angular@outils-equipe": true
  }
}
```

- La marketplace n'est prise en compte qu'après le dialogue de confiance du dossier. Un plugin dont la source est externe s'installe une fois par contributeur : `claude plugin install <plugin>@<marketplace> --scope project`.
- Un skill partagé a besoin d'un propriétaire : sans lui, il vieillit.
- Un skill peut exécuter du code : relisez-le avant de l'installer, comme une dépendance npm ou NuGet.

Composer des skills

Chaque skill fait une chose ; la puissance vient de la composition, par exemple la chaîne de Matt Pocock : `/grill-me` (clarifier l'intention) → `/to-spec` (écrire la spec) → `/to-tickets` (découper) → `/implement` (implémenter, test d'abord, avec `/tdd`). Pocock réserve `/grill-me` à ce qui n'est pas du code ; pour un changement de code, il conseille `/grill-with-docs`, la même interview, qui tient en plus le glossaire et les décisions du projet. Demain, votre ticket passe dans cette chaîne, avec `/grill-with-docs` en tête.

Qui fait quoi

Brique	Ce que c'est	Qui la déclenche	Exemple
Commande intégrée	Une fonction de Claude Code	Vous	`/context`, `/compact`
Skill	Un savoir ou une procédure, chargé à la demande	Vous ou Claude	`create-database-migration`
Sous-agent	Une instance de Claude à contexte isolé, qui rend un résultat	Claude ou vous	`security-reviewer` de Bénéfriches
MCP	Un connecteur vers un outil ou un service externe	Claude	context7 (documentation de bibliothèques), un navigateur, GitHub

Et les deux autres pièces du harness : **CLAUDE.md décrit** (contexte permanent : commandes, conventions, pièges), **le hook garantit** (exécution déterministe : formatage, protection, tests).

Quand chaque brique se charge :

Brique	Chargée	Ce qu'on y met
CLAUDE.md	À chaque session	Ce qui sert toujours : commandes, conventions, pièges
Règles `.claude/rules/`	Selon le dossier	Ce qui ne vaut que pour une partie du code
Skills	À la demande	Un savoir ou une procédure, utile à une tâche
Hooks	À coup sûr, hors du contexte	Ce qui doit arriver à chaque fois
Permissions	À chaque appel d'outil, hors du modèle	Ce que l'agent ne doit jamais faire

Les quatre zones du harness sont en place : le contexte, les automatismes, les capacités et les barrières. Demain : composer et paralléliser.

- Commande : le mot désigne ici les commandes intégrées. Les anciennes commandes personnalisées (`.claude/commands/*.md`) ont fusionné avec les skills : elles fonctionnent encore, et en cas de conflit de nom, le skill gagne. Les nouvelles s'écrivent en skills.
- Sous-agents : intégrés (dont Explore et Plan, en lecture seule, et general-purpose) ou définis dans `.claude/agents/*.md`. Demain, vous les ferez travailler en parallèle.
- MCP : se configure pour le projet dans `.mcp.json`. Par défaut, seuls les noms des outils et les instructions de chaque serveur sont chargés ; les définitions complètes arrivent à la demande (la recherche d'outils), sauf derrière une passerelle d'API (un `ANTHROPIC_BASE_URL` personnalisé) ou si cette recherche est désactivée. Un sous-agent peut même déclarer ses propres serveurs MCP (champ `mcpServers`) : c'est l'exercice MCP du J2. `/context` montre leur poids.

Des plugins utiles

Les plugins s'installent depuis la marketplace officielle `claude-plugins-official` : `/plugin install <nom>@claude-plugins-official`, ou `/plugin` puis l'onglet de découverte.

Plugin	Ce qu'il apporte
`skill-creator`	Créer, améliorer et évaluer des skills
`hookify`	Créer un hook à partir d'une phrase : « préviens-moi quand j'utilise rm -rf »
`typescript-lsp`	L'intelligence de code TypeScript et JavaScript : diagnostics, navigation. Binaire à installer : `typescript-language-server`
`csharp-lsp`	La même chose pour C#. Binaire : `csharp-ls`
`jdtls-lsp`	La même chose pour Java. Binaire : `jdtls`
`pyright-lsp`	La même chose pour Python. Binaire : `pyright-langserver`

```
/plugin install skill-creator@claude-plugins-official
/plugin install hookify@claude-plugins-official
/plugin install typescript-lsp@claude-plugins-official
/plugin install csharp-lsp@claude-plugins-official
/plugin install jdtls-lsp@claude-plugins-official
/plugin install pyright-lsp@claude-plugins-official
```

Un plugin LSP ne contient pas le serveur de langage : installez d'abord le binaire indiqué et vérifiez qu'il est dans le PATH du terminal où vous lancez `claude`. Pour TypeScript : `npm install -g typescript-language-server typescript`. Un plugin peut exécuter du code : regardez ce qu'il installe avant de valider.

J2 matin — Récap

Composer & piloter la boucle. Au J1, vous avez configuré le harness ; au J2 matin, vous vous en servez sur un ticket qui tient la journée, sur votre projet ou sur Tailspin Toys, le projet d'entraînement : composer un workflow, choisir votre niveau de supervision, lancer une loop qui s'arrête toute seule. À la formule du J1 (CLAUDE.md décrit, les hooks garantissent, les skills complètent à la demande), ajoutez deux verbes : les workflows enchaînent, les loops tournent jusqu'à une condition que vous avez écrite.

Découvrir un projet avec Claude

Une base de code inconnue, lundi chez un client comme vendredi sur Tailspin Toys : faites-vous la présenter par Claude, sans rien modifier. D'abord à votre façon, ce que vous demanderiez à un collègue le premier jour ; puis avec un prompt précis, qui donne à tous la même base :

```
Ne modifie rien. Présente-moi ce projet en cinq lignes : ce qu'il fait,
sa stack, comment on lance les tests. Puis résume chaque ticket de
backlog/ en une ligne : ce qui change pour l'utilisateur.
```

Ce prompt applique trois des quatre verbes du J1 : référencer (un dossier nommé, `backlog/`), contraindre (« Ne modifie rien »), structurer (cinq lignes, puis une ligne par ticket). Sur votre projet, remplacez `backlog/` par votre outil de tickets.

Tailspin Toys est un site fictif de financement participatif de jeux, que GitHub utilise pour ses ateliers Copilot : un catalogue de 21 jeux, en Astro 7 et TypeScript, avec une base SQLite intégrée à Node ; rien d'autre à installer que Node. Le fork du J2 (`denneulin/tailspin-toys`) est préparé pour Claude Code : un CLAUDE.md, quatre règles dans `.claude/rules/`, les tests, le lint et les types autorisés sans permission, des worktrees qui partent du commit courant, et neuf tickets dans `backlog/`. Interrogé, Claude lit le README et le CLAUDE.md, et rend la stack, le flux des données, ce harness et, de lui-même, le backlog. Pour l'installer (Git et Node 24) :

```
git clone https://github.com/denneulin/tailspin-toys.git
cd tailspin-toys
npm ci
npm run dev        # http://localhost:4321
```

Le ticket du jour

Le J2 tourne autour d'un seul ticket, le ticket fil rouge : sa spec et ses tickets le matin, une loop sur le premier avant le déjeuner, les suivants en parallèle l'après-midi. Un bon candidat dans votre backlog : une demi-journée à deux jours de travail, découpable en deux ou trois morceaux, vérifiable par des tests, montrable. Écrivez-le en une phrase : ce qui change pour l'utilisateur, pas comment le coder.

- Trop petit (un libellé, une couleur) : rien à découper ni à paralléliser. Élargissez-le.
- Trop gros : gardez-en une tranche, qui se livre et se vérifie seule.
- Confidentiel : décrivez-le en termes génériques.
- Sur Tailspin, la 07 (filtrer les jeux par catégorie et par éditeur) était proposée à tous : son ticket dit comment la livrer, avec deux tickets à lancer en parallèle l'après-midi.

Composer plutôt qu'adopter

	Adopter une méthode	Composer un workflow
Ce que c'est	Une méthode clé en main (GSD, BMAD, Spec-Kit…) qui possède le processus : ses étapes, ses gabarits, ses agents	De petits skills, une responsabilité chacun, enchaînés par vous ou par un script
Ce qu'on y gagne	Un démarrage rapide	La main sur chaque étape
Quand une étape ne vous va pas	Difficile à corriger : c'est leur processus, pas le vôtre. Vous forkez la méthode, ou vous vivez avec.	Vous modifiez un `SKILL.md` : votre copie dans `.claude/skills/`, pas celle du plugin, remplacée à chaque mise à jour

C'est la position de Matt Pocock dans le README de ses skills : les méthodes possèdent le processus et vous retirent le contrôle ; ses skills sont petits, faciles à adapter et composables. Une méthode s'adopte en bloc ; un workflow se compose. Ce n'est pas un dogme : une méthode démarre vite, et on peut lui emprunter de bonnes idées (voir « Doser »).

La spec, source de vérité

Pour Birgitta Böckeler (martinfowler.com, octobre 2025), une spec est un artefact structuré, en langage naturel, orienté comportement, qui décrit une fonctionnalité et guide l'agent. Pas un cahier des charges de quarante pages : un fichier Markdown que l'agent lit. Elle distingue trois niveaux.

Niveau	La spec…	Exemple
Spec-first	est écrite avant, et sert à la tâche en cours	Spec-Kit, Kiro
Spec-anchored	est gardée après la tâche, et sert à faire évoluer et à maintenir la fonctionnalité	Une spec versionnée à côté du code
Spec-as-source	est la seule chose qu'on édite ; le code en découle	Tessl (en bêta privée en 2025)

Au J2 matin, vous faites du spec-first : la chaîne écrit la spec dans le repo, sur votre branche de travail. La garder dans git, à côté du code (spec-anchored), est une décision d'équipe.

La recommandation de la documentation Claude Code (page Best practices), pour une grosse fonctionnalité :

1. Laissez Claude vous interviewer.
2. Faites-lui écrire une spec complète dans un fichier (`SPEC.md`).
3. Ouvrez une nouvelle session pour l'exécuter, avec un contexte propre.

L'interview a rempli la fenêtre de questions, d'hésitations et d'explorations ; la spec en est la version distillée. Après un `/clear`, tout ce qui a été dit dans la conversation a disparu ; la spec, elle, est sur le disque. Une spec écrite survit au `/clear` ; une conversation, non.

Selon la même page, une bonne spec est autonome (« self-contained ») : elle nomme les fichiers et les interfaces concernés, dit ce qui est hors périmètre, et se termine par une étape de vérification de bout en bout. Pour la 05 de Tailspin, « un résumé du catalogue sur l'accueil : nombre de jeux, note moyenne » : elle nomme la fonction d'accès aux données qui calcule le résumé et la page d'accueil qui l'affiche, dit que la page par éditeur (la 04) est hors périmètre, et se termine par « les tests, le lint et les types passent, et l'accueil affiche le nombre de jeux et la note moyenne ».

La chaîne de Matt Pocock

Quatre skills qui mènent d'une idée floue à des commits. L'interview, la spec et les tickets s'enchaînent dans une même conversation, sans `/clear` entre eux ; la spec et les tickets restent sur le disque, et chaque ticket s'implémente ensuite en contexte frais.

Skill	Ce qu'il fait	Ce qu'il produit
`/grill-with-docs`	Claude vous interviewe, par rounds, jusqu'à ce que l'intention soit claire, et tient au passage le glossaire du projet et ses décisions d'architecture.	La conversation : ses questions, ses recommandations, vos réponses ; plus `GLOSSARY.md` et, parfois, une ADR dans `docs/adr/`
`/to-spec`	Pas d'interview : il synthétise la conversation. Il explore le code, vous propose ses points de test (seams), là où les tests attaqueront le code, et prévoit de vous les faire valider.	`spec.md`, au gabarit fixe (ci-dessous)
`/to-tickets`	Il découpe la spec en tranches verticales, vous présente le découpage et vous interroge (granularité, dépendances, fusions) avant de publier.	Un fichier par ticket, avec sa ligne « Blocked by » et ses critères d'acceptation en cases à cocher
`/implement`	Un ticket par invocation : il pilote `/tdd` aux points de test convenus, puis son `/code-review` avant de committer.	Des commits (J2 après-midi)

- Ces quatre skills sont invoqués par l'utilisateur (`disable-model-invocation: true`) : Claude ne les lance jamais seul. Ils peuvent appeler des skills invoqués par le modèle (`tdd`, `code-review`, `grilling`, `domain-modeling`), jamais un autre skill invoqué par l'utilisateur. C'est donc vous qui enchaînez les étapes : vous êtes le séquenceur.
- Deux régimes. Pendant la conception (`/grill-with-docs`, `/to-spec`, `/to-tickets`), vous répondez et vous validez. Pendant l'implémentation, ce sont les tests et la revue qui vérifient. Vous retrouverez cette frontière dans les niveaux de supervision.
- `/grill-me`, vu au J1, lance la même interview, le skill `grilling`, sans glossaire ni ADR. Matt Pocock le réserve à ce qui n'est pas du code, et recommande `/grill-with-docs` dès qu'il y a une base de code : il y ajoute `domain-modeling`, qui tient le glossaire et les ADR.
- Attention au nom : dans le menu, le `/code-review` de Pocock s'appelle `/mattpocock-skills:code-review` ; `/code-review` tout court lance la relecture intégrée à Claude Code, un autre outil.

`/grill-with-docs` : une interview par rounds

Un vrai round, sur la 05 de Tailspin, le résumé du catalogue sur l'accueil (Claude Code 2.1.301, Opus 5.5). La commande :

```
/grill-with-docs Je veux livrer le ticket backlog/05-catalog-summary.md : un résumé du catalogue sur l'accueil.
```

Le début du premier round :

```
Faits établis (pas besoin de vous les demander)
- Le catalogue compte 21 jeux, tous notés. (…) La moyenne réelle
  est de 3,905.
- Il n'y a ni GLOSSARY.md ni docs/adr/. Je les créerai au fil de
  l'entretien, quand un terme sera tranché.
- (…)
- D'après .claude/rules/donnees.md, une fonction d'accès aux données
  reçoit db en premier paramètre et se teste sur une base en mémoire,
  sans mock.

Q1 — Le mot « catalogue » : le ticket dit catalog, le sous-titre de
l'accueil dit collection, et le titre de section Featured Games.
Le catalogue, est-ce tous les jeux de la base, sans exception (…) ?
Et faut-il écarter collection comme synonyme ?

→ Oui. Catalogue = l'ensemble des jeux proposés sur le site,
c'est-à-dire tous les jeux de la base. On écarte collection et
inventaire. « Featured Games » reste un libellé d'interface et
n'entre pas dans le glossaire.
```

- Chaque question arrive avec sa recommandation, la ligne qui commence par →. Le round complet comptait six questions : le mot « catalogue » ; ce qu'est un jeu noté (une note de 0, une note de 7) ; où se fait le calcul ; ce que renvoie la fonction et qui arrondit ; les cas limites à l'écran (aucun jeu, aucun jeu noté, le singulier) ; le placement et le balisage.
- Pour valider toutes les recommandations d'un round, dites-le en toutes lettres : « j'accepte toutes tes recommandations ». Vous pouvez aussi répondre question par question : « Q4 : c'est la page qui arrondit. » La version de la marketplace de Matt formule ses questions pour qu'un « oui » accepte la recommandation ; la phrase complète marche toujours. Ce « oui » de round n'est pas celui de la fin de l'interview : voir le piège, ci-dessous.
- La Q1 est la marque de `/grill-with-docs` : il traque les mots flous ou en double, et vous les fait trancher. Le terme retenu part aussitôt dans `GLOSSARY.md`, à la racine du repo : un glossaire, sans détail d'implémentation, que les sessions suivantes liront. Il ne propose une ADR, dans `docs/adr/`, que pour une décision difficile à défaire, surprenante sans contexte, et issue d'un vrai compromis. Il confronte aussi ce que vous dites au code : « Vous dites que tous les jeux sont notés ; le schéma permet une note vide. Lequel a raison ? »
- Les questions suivent un arbre de décision : Claude ne pose que celles dont les prérequis sont réglés (ce que Pocock appelle la frontière). Chaque réponse débloque les questions du round suivant.
- Les faits, Claude les cherche lui-même dans le code, parfois par un sous-agent qui explore avant le premier round. Les décisions sont pour vous. Il ne vous demande pas où se calcule la note ; il vous demande ce qu'est un jeu noté.
- C'est fini quand il ne reste plus de question, et Claude n'agit pas avant votre confirmation. Avant de passer à la spec, demandez-lui : « Quelles décisions as-tu prises à ma place ? » Il liste ce qu'il a supposé sans le demander ; c'est souvent là que se cachent les cas limites.
- Pourquoi ça marche : pour Pocock, le premier mode d'échec avec un agent, c'est « l'agent n'a pas fait ce que je voulais ». Un défaut d'alignement, pas de code : l'interview le traite avant la première ligne.

> **Le piège.** À la fin de l'interview, Claude résume ce qu'il a compris et demande confirmation avant de toucher au code. Si vous répondez « oui, vas-y », il code. Pour enchaîner sur la spec, ne répondez pas « oui » : tapez `/to-spec`, ou précisez « ne touche pas au code ». S'il a déjà commencé : Esc, puis `/rewind` pour revenir au point d'avant (il ne suit pas les modifications faites par des commandes Bash), ou « Annule ces modifications » ; `git status` pour vérifier.

Installer la chaîne

```
# Au J2 : la marketplace de Matt (affichée 1.3.1)
/plugin marketplace add mattpocock/skills
/plugin install mattpocock-skills@mattpocock

# Ou bien, à la place : la marketplace officielle (affichée 1.2.3)
/plugin install mattpocock-skills@claude-plugins-official

# Puis une fois par repo, avant les autres skills :
/setup-matt-pocock-skills
```

- Au J2, une seule voie, la marketplace de Matt, pour que toute la salle ait la même version. Chez vous, la marketplace officielle est la voie que Matt Pocock recommande pour Claude Code. L'une ou l'autre, pas les deux : sinon, chaque skill est en double. `/plugin install` demande la portée de l'installation : pour vous seul suffit.
- La marketplace officielle épingle un commit du 18/09/2026, que Claude Code affiche en 1.2.3 ; elle se met à jour quand ce numéro change. Celle de Matt sert la dernière version de sa branche principale (au 14/10/2026, un commit du 09/10, affiché 1.3.1) et ajoute notamment `/implement-spec`. `/grill-me` et `/to-spec` sont identiques dans les deux versions ; `grilling`, `/implement`, `/tdd` et `/code-review` diffèrent légèrement, et le glossaire s'appelle `CONTEXT.md` dans la 1.2.3, `GLOSSARY.md` dans la 1.3.1.
- Un plugin peut exécuter du code : lisez ce que vous installez. Le dépôt est open source (licence MIT), et le `SKILL.md` de `grill-with-docs` tient en une ligne, qui appelle deux skills, `grilling` (l'interview) et `domain-modeling` (le glossaire et les ADR).

`/setup-matt-pocock-skills`, une fois par repo, avant les autres skills. Il explore le repo, puis il vous pose jusqu'à trois questions : le gestionnaire de tickets (GitHub, GitLab, fichiers Markdown locaux, ou un autre outil que vous décrivez en un paragraphe, Jira par exemple), les labels de tri (gardez ceux par défaut) et l'organisation des docs de domaine, cette dernière seulement sur un monorepo. Il écrit trois fichiers dans `docs/agents/` (`issue-tracker.md`, `domain.md`, `triage-labels.md`) et ajoute un bloc « ## Agent skills » à votre CLAUDE.md, ou à votre AGENTS.md s'il n'y a pas de CLAUDE.md. Il modifie donc votre repo : faites-le sur une branche. Sur Tailspin, c'est déjà fait et committé : le bloc est à la fin du CLAUDE.md.

Choisissez **fichiers Markdown locaux** : rien à configurer, aucun droit à demander, tout reste sur votre poste. La spec et les tickets arrivent dans `.scratch/`, un dossier par fonctionnalité :

```
.scratch/
└── <nom-de-la-fonctionnalité>/
    ├── spec.md
    └── issues/
        ├── 01-<titre>.md     numérotés dans l'ordre des dépendances ;
        └── 02-<titre>.md     une ligne « Blocked by », une ligne « Status »
```

Si `.scratch/` n'est pas dans votre `.gitignore`, vos specs partent avec le code : c'est une décision d'équipe (spec-first ou spec-anchored). Pour l'Atelier 3, committez-les sur votre branche de travail, avec `GLOSSARY.md` et `docs/adr/`.

La branche. Dès l'interview, Claude écrit dans le repo (`GLOSSARY.md`, parfois `docs/adr/`, puis `.scratch/`) : créez d'abord une branche au nom de votre ticket. C'est votre branche de travail : tout ce que vous produisez dans la journée y va, les worktrees de l'après-midi partent de ses commits, et on y fusionne leurs branches. Le départ de la chaîne, au J2 :

```
git switch -c <votre-ticket>     # 07-filtres pour la 07
/model opus
/grill-with-docs <votre ticket : le chemin du fichier, ou votre phrase>
```

Le modèle. Interviewer, spécifier, découper, c'est du jugement : la chaîne tourne sur Opus, le modèle par défaut de Claude Code (`/model opus` pour le vérifier). L'implémentation, elle, passe sur Sonnet : voir « Un modèle par phase », au J2 après-midi.

`/to-spec` : les points de test, puis la spec

Avant d'écrire, `/to-spec` vous propose ses points de test (seams), là où ses tests attaqueront le code : une fonction, une route, une page. Pocock les veut placés le plus haut possible, et le moins nombreux possible ; l'idéal, c'est un seul. Le skill prévoit de vous les faire valider, mais à la répétition, `/to-spec` tapé seul a écrit la spec directement : tapez une consigne après la commande : elle complète le skill.

```
/to-spec Dis-moi d'abord où tu comptes tester, et attends mon accord avant d'écrire.
```

Puis il écrit `spec.md` dans `.scratch/<fonctionnalité>/`, toujours avec les mêmes sections :

Section	Ce qu'on y trouve
Problem Statement	Le problème, du point de vue de l'utilisateur
Solution	Ce qui change, en quelques phrases
User Stories	Une liste longue et numérotée, cas limites compris
Implementation Decisions	Les modules, les interfaces, les schémas, les contrats d'API ; pas de chemins de fichiers ni de code, qui vieillissent vite
Testing Decisions	Les points de test que vous avez validés ; ce qu'est un bon test
Out of Scope	Ce qu'on ne fait pas
Further Notes	Les précisions qui restent

Petite différence d'école : la doc de Claude Code veut une spec qui nomme les fichiers ; Pocock nomme les modules et les interfaces, pas les chemins. Une spec longue, c'est le gabarit : lisez d'abord Implementation Decisions et Out of Scope. Ce qui manque à Out of Scope, faites-le ajouter : « Ajoute à Out of Scope : … ». Pour les questions restées sans réponse, il peut compléter avec ses propres recommandations : relisez ce passage, et tranchez chaque point.

Les règles des tickets

- **Des tranches verticales** (« tracer bullets ») : chaque ticket traverse toutes les couches, se démontre ou se vérifie seul, et tient dans une fenêtre de contexte fraîche. Sur Tailspin, une tranche traverse la fonction de `src/lib/` et son test Vitest, la page ou le composant Astro, et la route si elle enregistre quelque chose.
- **Jamais par couche** (« 01 la fonction de filtrage, 02 l'interface ») : aucun ticket ne se vérifie seul. La phrase à taper : « Une tranche verticale par ticket, testable seule. »
- **Une ligne « Blocked by » par ticket** : le graphe des dépendances dit, avant de lancer le moindre agent, ce qui pourra partir en parallèle.
- **Un premier ticket qui en débloque deux**, pour paralléliser ensuite : demandez-le à `/to-tickets`. Le premier part seul (au J2, dans une loop, avant le déjeuner) ; puis les deux suivants partent en même temps, un worktree chacun.
- **Un refactoring préalable d'abord**, s'il le faut : `/to-tickets` le cherche en premier (« Make the change easy, then make the easy change »).
- **L'exception, le refactoring mécanique large** (renommer une colonne partout) : il se fait en expand–contract. On ajoute le nouveau (l'expand, seul), on migre les appelants par lots, on retire l'ancien. Les trois phases se suivent ; seuls les lots de migration, qui ne dépendent que de l'expand, peuvent partir en parallèle.
- **Trop de tickets** (six ou sept pour la 07 de Tailspin) : « Fusionne en quatre tickets au plus. »

Chaque ticket est un fichier de `.scratch/<fonctionnalité>/issues/`, numéroté dans l'ordre des dépendances : ce qu'il faut construire, sa ligne « Blocked by », sa ligne « Status », et ses critères d'acceptation en cases à cocher. Ces cases, votre condition `/goal` les reprendra.

Sur la 07 de Tailspin (filtrer les jeux par catégorie et par éditeur), le ticket a une section « Delivery » qui dit comment le livrer. Aux trois dernières répétitions, `/to-tickets` l'a suivie sans qu'on le demande :

Ticket	Bloqué par
01 Une catégorie, de bout en bout	Rien
02 L'éditeur	01
03 Plusieurs catégories	01
04 La combinaison des filtres, en tranche à part	02, 03

Sans cette section, deux répétitions sur trois donnaient un graphe linéaire ; sans « en tranche à part », une sur trois fondait la combinaison dans le 02. Sur la 05, il a gardé deux tickets : 01 le nombre de jeux, puis 02 la note moyenne, bloquée par 01. Un graphe linéaire n'est pas un échec : c'est une information, qu'on a avant de lancer le moindre agent. S'il est linéaire alors que le ticket semblait découpable, demandez à `/to-tickets` pourquoi : parfois c'est justifié (un refactoring large) ; parfois, un ticket préalable qui rend le changement facile débloque tout le reste.

Doser : du plus simple au plus outillé

Toutes les tâches ne méritent pas une interview. Quatre marches, chacune plus coûteuse que la précédente : prenez la plus basse qui suffit.

Marche	Comment
1. Plan mode	Shift+Tab, ou `/plan <description>` : Claude lit et propose un plan, sans modifier vos fichiers. Ctrl+G ouvre ce plan dans votre éditeur, pour le corriger avant de lancer quoi que ce soit.
2. Interview → spec (brainstorming → PRD)	`/grill-with-docs` puis `/to-spec` : la spec tient le rôle du PRD, le document d'exigences produit. Sans plugin, un prompt fait l'essentiel : « Interviewe-moi sur cette fonctionnalité, puis écris une spec complète dans SPEC.md », puis une nouvelle session pour l'exécuter.
3. Votre workflow composé	Vos skills, vos fichiers, vos tests : ce que vous esquissez dans le plan de composition.
4. Une méthode clé en main	Spec-Kit, Kiro, BMAD, GSD Core, Superpowers : plus de cérémonie, plus de coût.

Si le diff tient en une phrase, pas de plan. La documentation de Claude Code le dit en substance : le plan a un coût ; il sert quand l'approche est incertaine, quand plusieurs fichiers changent, ou quand on connaît mal le code. Elle recommande quatre phases : Explore (en Plan mode, lire sans modifier), Plan (demander un plan détaillé, le retoucher avec Ctrl+G), Implement (sortir du Plan mode et coder en vérifiant contre le plan), Commit.

- Changer la couleur d'un bouton ; en .NET, renommer une propriété d'un DTO utilisée par un seul contrôleur : une phrase, pas de plan.
- Ajouter la pagination et le tri à l'endpoint de recherche des clients : un Plan mode au minimum, une interview si les règles de tri ne sont pas claires.
- Un export CSV avec un filtre : une interview.
- Un ticket comme la 07 de Tailspin, plusieurs tranches et des décisions à prendre : la chaîne complète.

Les méthodes clé en main et leurs limites

État au 14/10/2026 :

Méthode	Le principe	Avec Claude Code	Ce qu'on lui reproche
Spec-Kit (GitHub)	Une constitution par projet, puis specify → plan → tasks → implement → converge	Oui, en skills	Une même cérémonie pour des tâches de toutes tailles (Böckeler) ; le retour du « waterfall » (Marmelab, 11/2025, sur le spec-driven en général) ; pour du travail ponctuel, le plan mode peut suffire, reconnaît son mainteneur principal (05/2026)
Kiro (AWS)	Un agent (IDE, CLI, web) qui produit requirements, design et tasks ; variante Quick Spec	Non : outil autonome	Trop verbeux pour un petit bug (Böckeler) ; Kiro reconnaît le poids du processus et lance un mode rapide, aujourd'hui Quick Spec (billet du 12/05/2026)
BMAD	Des agents analyste, PM, architecte, dev et UX ; la méthode s'ajuste à la taille du travail	Oui : plugin ou `npx bmad-method install`	Lourdeur, courbe d'apprentissage raide (Hicron, 05/2026)
GSD Core	Discuss → plan → execute → verify → ship ; le travail lourd en sous-agents à contexte frais	Oui	Des difficultés sur des besoins ambigus ou mouvants (Hicron)
Superpowers	Des skills composables : brainstorming, plans, TDD, review	Oui, marketplace officielle	Un coût en tokens ; redondant avec le Plan mode et les sous-agents natifs (MCP.Directory, 06/2026)

À toutes, on peut reprocher des plans très longs, que personne ne relit. Elles changent vite, aussi : en 2026, le dépôt d'origine de GSD a été archivé et remplacé par GSD Core, et BMAD annonce sa version 7 pour le 16 octobre. Un workflow composé avec vos skills ne change que quand vous le décidez. On pioche des idées dans les méthodes ; on ne les adopte pas en bloc.

Ce qu'elles ont toutes en commun : des skills, des gabarits Markdown, parfois des sous-agents. Ce qu'on en garde :

Principe	La brique
Des skills atomiques	Une responsabilité chacun : `/grill-with-docs` n'écrit pas la spec, `/to-spec` n'interviewe pas
Un fichier court entre deux étapes	La spec, les tickets, le plan : il se relit, se versionne, survit au `/clear`
Un contexte frais par phase	La conception dans une conversation, chaque implémentation dans une autre : `/clear`, une nouvelle session, un sous-agent
Des vérifications automatiques	Les tests, les hooks, `/goal`
Un séquenceur	Vous, un skill qui orchestre, un script (`claude -p`)

Cet assemblage, c'est votre harness. Le critère d'une bonne brique : une responsabilité, un fichier en sortie, et lançable à la main. Si une étape ne marche pas en interactif, elle ne marchera pas dans un script.

Superviser : in, on, outside the loop

Kief Morris (martinfowler.com, mars 2026) part de deux boucles : celle du pourquoi (transformer des idées en logiciel) et celle du comment (produire le code, les tests, l'outillage). La question : où se place l'humain par rapport à la seconde ?

Position	Vous...	Remarque
In the loop	contrôlez chaque sortie, en gardien de la boucle : chaque diff est relu ligne à ligne	
On the loop	améliorez le harness (specs, contrôles qualité, consignes de workflow) qui produit les sorties, au lieu de les inspecter une à une	La position que Morris recommande
Outside the loop	restez sur le pourquoi ; les agents gèrent le comment	Proche du vibe coding ; Dex Horthy l'a essayé en 2025, et son équipe a dû réécrire from scratch (voir « Le coût humain », J2 après-midi)

C'est le spectre de la délégation, du développeur à l'orchestrateur : in the loop, vous relisez chaque sortie ; on the loop, vous orchestrez des agents à travers le harness. Plus vous déléguez, plus votre travail passe de la relecture des sorties au pilotage du harness.

Quand l'agent se trompe, la question n'est plus « comment je corrige ce code ? » mais « qu'est-ce qui, dans le harness, a laissé passer ça ? ». Exemple, le code généré « peu propre » : in the loop, vous nettoyez chaque composant à la main, à chaque fois ; on the loop, vous ajoutez les variables inutiles aux pièges de votre skill de revue, ou une règle de lint.

Le niveau dépend de la tâche

Étape ou tâche	Niveau	Pourquoi
Conception : spec, découpage, critères	in the loop	Les décisions sont les vôtres
Implémentation d'un ticket bien spécifié et testé	on the loop	Les tests, les hooks et `/goal` vérifient pour vous
Décider du merge	in the loop	Sur preuves : tests verts, rapport, relecture en contexte frais
Prototype jetable	outside, à la rigueur	Rien à maintenir
Migration de base de production, sécurité	in the loop	L'erreur coûte trop cher

Monter d'un cran demande un harness plus solide, pas plus de courage. Sans tests sur la zone, vous n'êtes pas on the loop : vous êtes outside sans le savoir.

Quand vous rentrez dans la boucle

Chaque intervention est un symptôme : corrigez la cause, à l'un de ces quatre endroits, plutôt que la sortie.

Ce qui a cloché	Ce qu'on corrige	Exemple
Il manquait une vérification : un test, un hook, une condition	Le harness	L'agent a rendu un code qui ne marche pas, et rien ne l'a arrêté : ajoutez le test, le hook ou la clause de condition qui aurait échoué.
La spec avait un trou	La spec, puis on relance	L'agent a choisi un format de date que vous ne vouliez pas. Corrigez la spec : sinon, le prochain agent refera le même choix.
Le code résiste : module trop couplé, pas de point d'entrée testable	Le code	L'agent tourne en rond parce que la tâche est difficile pour n'importe qui : rendez d'abord le changement facile.
Une vraie erreur de l'agent	On relance, on découpe plus fin	Ça arrive ; mais posez-vous d'abord les trois premières questions.

Corriger la sortie, c'est rester dans la boucle pour toujours. Exemple .NET : Claude écrit un endpoint de recherche sans limite de taille de page. In the loop, vous ajoutez la limite à la main ; on the loop, vous ajoutez à la spec « la taille de page est plafonnée » et un test qui le vérifie, et le prochain ticket sur un endpoint en profite. Nuance : sur une tâche unique, corriger à la main est légitime, c'est plus rapide ; l'investissement dans le harness ne paie que sur ce qui se répète. Cette grille sert à l'étape 5 de l'Atelier 3, pour classer vos tentations d'intervenir (voir « L'Atelier 3, geste par geste », au J2 après-midi).

Rendre la vérification obligatoire

« Claude stops when the work looks done » (documentation, Best practices) : Claude s'arrête quand le travail a l'air fini. Pour qu'il s'arrête quand le travail est fini, donnez-lui un moyen de vérifier. Quatre marches, de la plus légère à la plus lourde :

Marche	Comment	Exemple
1. Dans le prompt	Une consigne	« lance les tests et corrige »
2. Sur la session	Une condition, évaluée à chaque tour	`/goal`
3. En barrière déterministe	Un script	Le hook Stop du J1 (`tests-verts.mjs`)
4. Par un second avis	Une relecture en contexte frais	Un sous-agent qui relit : le `/code-review` de Pocock, le relecteur de votre pipeline

« Each step trades setup for attention » : chaque marche échange du temps d'installation contre de l'attention. Plus vous lâchez la main, plus la vérification doit être automatique.

Les loops

Une loop, c'est un cycle d'exécution de l'agent, défini par ce qui le déclenche et ce qui l'arrête. La typologie vient d'Anthropic (« Loop engineering: Getting started with loops », juin 2026).

Type	Lancée par	S'arrête quand	Dans Claude Code
Turn-based	Votre prompt	Claude juge que c'est fini	Une conversation normale
Goal-based	Une condition	La condition est atteinte (ou le plafond dépassé)	`/goal`
Time-based	Un intervalle	Vous l'arrêtez, ou le travail aboutit	`/loop`, `/schedule`
Proactive	Un événement ou un planning	Chaque tâche, quand son but est atteint ; la routine, quand vous la coupez	Routines, tâches planifiées du desktop, GitHub Actions

La vraie question : qui décide que c'est fini ? Dans une loop turn-based, c'est le modèle qui a travaillé. Avec `/goal`, c'est un autre modèle, à partir d'une condition que vous avez écrite.

`/goal` : une condition, un évaluateur

`/goal <condition>` fixe la condition de fin et démarre un tour tout de suite : la condition sert aussi de consigne. Un seul goal par session ; un nouveau remplace l'ancien. L'indicateur `◎ /goal active` s'affiche, avec la durée. Après chaque tour, Claude Code envoie la condition et la conversation à un petit modèle rapide, Haiku par défaut : l'évaluateur. Il rend un verdict, avec une raison courte.

Verdict	Ce qui se passe
Pas encore	Claude continue ; la raison lui sert de consigne pour le tour suivant.
Atteint	Le goal s'efface ; une entrée « achieved » apparaît dans la transcription.
Impossible	Le goal s'efface, avec la raison.

L'évaluateur ne lance aucune commande et ne lit aucun fichier : il juge seulement ce que Claude a affiché dans la conversation. D'où la règle d'écriture d'une condition, plus bas.

Commande ou raccourci	Effet
`/goal <condition>`	Fixe la condition et démarre un tour
`/goal`	Le statut : la condition, la durée, le nombre de tours évalués, les tokens dépensés, la dernière raison
`/goal clear`	Efface le goal ; un `/clear` l'efface aussi
Ctrl+O	Affiche la raison de chaque verdict
`claude -p "/goal …"`	En non interactif : la loop tourne jusqu'au bout en une seule invocation

- **Le mode de permission ne change pas.** En Auto, le mode de départ, les tours s'enchaînent sans vous ; en Manual ou en Accept edits, Claude vous demande encore avant les outils non autorisés. La doc le résume : le mode Auto supprime les demandes par outil, `/goal` supprime les demandes par tour.
- **Une loop qui tourne sans vous**, au J2 pendant le déjeuner : vérifiez d'abord que la barre d'état affiche « ▸▸ auto mode on » ; sinon, Shift+Tab jusqu'à l'afficher. S'il y est déjà, n'y touchez pas : depuis Auto, Shift+Tab passe en Manual, et la loop attendrait la première demande de permission. Puis la session reste ouverte, l'ordinateur branché, sans mise en veille : rien n'avance tant qu'il dort. Et pas de `/clear`, qui efface le goal.

- **Deux sécurités intégrées.** Si Claude répond à l'évaluateur sans progresser (aucun outil pendant plusieurs tours), Claude Code arrête la loop et vous rend la main, le goal toujours actif. Une erreur que vous devez corriger vous-même (authentification, crédit épuisé, contexte saturé, modèle indisponible) efface le goal.
- **Le mécanisme.** `/goal` est un hook Stop à base de prompt, limité à la session ; comme les hooks, il dépend de la confiance accordée au dossier. En reprise de session (`--continue`, `--resume`), le goal actif est restauré, compteurs remis à zéro.

`/goal` ou le hook Stop du J1 ? Le juge : un petit modèle qui lit la conversation pour `/goal`, votre script pour le hook. La portée : `/goal` vaut pour la session ; le hook est versionné dans le repo et vaut pour toute l'équipe.

« `/goal` à blanc »

Avant d'écrire une bonne condition, essayez-en une mauvaise. Sur votre branche, d'abord un `/clear` : sans lui, Claude a vos tickets en tête et pourrait se mettre à coder.

```
/clear
/goal npm run test:unit passe ; sinon arrête-toi après 2 tours
```

Sur Tailspin, Claude lance les tests, les 39 passent déjà, et l'évaluateur rend « atteint » au premier tour : aucune ligne de code n'a été écrite, et pourtant la condition est remplie. Une condition déjà vraie avant le travail ne prouve rien. Sur un projet sans tests, ou avec une commande qui échoue, c'est l'autre leçon : l'évaluateur rend « pas encore », et le plafond arrête Claude ; ou il rend « impossible », et le goal s'efface avant.

Écrire une condition d'arrêt

Floue : « /goal répare le module commandes ». Réparé selon quel critère ? Qu'est-ce qui le prouve ? Ni vous ni l'évaluateur ne pouvez trancher en lisant la sortie.

Vérifiable, en .NET :

```
/goal `dotnet test --filter Category=Orders`
  passe sans échec ;
  aucun fichier de test existant n'est modifié ;
  sinon arrête-toi après 15 tours.
```

La même, en Angular :

```
/goal `npm test -- --watch=false` passe sans échec ;
  aucun fichier *.spec.ts n'est modifié ;
  sinon arrête-toi après 15 tours.
```

Le modèle du J2, sur Tailspin, pour le premier ticket d'un graphe (sur votre projet, vos commandes de tests, de lint ou de build) :

```
/goal <le nouveau test du ticket, nommé> passe,
  et npm run test:unit, npm run lint et npm run typecheck passent ;
  aucun autre fichier de test n'est modifié, ce que montre git status ;
  sinon arrête-toi après <n> tours.
```

Les retours à la ligne ne servent qu'à la lecture : collez la condition sur une seule ligne. Pour l'écrire, partez de la commande de vérification de votre plan de composition et des critères d'acceptation du ticket : chaque case à cocher vérifiable peut devenir une clause.

Ingrédient (doc `/goal`)	Dans l'exemple .NET
Un état final mesurable	passe sans échec
Une vérification énoncée	La commande `dotnet test`, dont la sortie s'affiche
Les contraintes qui comptent	Aucun fichier de test existant n'est modifié
Et un plafond	15 tours

- Un état final se mesure : un résultat de tests, un code de sortie, un nombre de fichiers, une file vide. Une vérification se nomme : « `npm test` sort en 0 », « `git status` est propre ».
- Le plafond est une clause de la condition elle-même : Claude rend compte de sa progression, et l'évaluateur juge aussi cette clause.
- La contrainte « aucun test n'est modifié » protège la loop : sans elle, le chemin le plus court vers « les tests passent » consiste parfois à modifier le test.
- Les conditions .NET et Angular valent pour une correction : au départ, la suite est rouge. Pour une fonctionnalité, elle est déjà verte avant le travail : c'est « `/goal` à blanc », atteint au premier tour sans une ligne écrite. Nommez alors le test du ticket, par exemple, pour la 05 de Tailspin : « un nouveau test Vitest prouve que la fonction du résumé renvoie le nombre de jeux et la note moyenne ».
- Tout doit être visible dans la sortie. « `dotnet test` passe » marche parce que Claude lance les tests et que le résultat s'affiche ; pour « aucun fichier de test n'est modifié », il faut que Claude le montre, par exemple avec un `git status` demandé dans la condition. À la répétition, sur Tailspin, une condition de ce genre l'a poussé à le faire de lui-même.
- Les pièges fréquents : « le code est propre », « le filtre fonctionne » sont invérifiables ; remplacez-les par une commande de lint qui sort en 0, ou par un test qui prouve le filtre et la commande qui le lance. L'écran, c'est vous qui le regardez. Le plafond oublié est l'oubli le plus courant.
- Pas de test sur la zone : la condition commence par « un test qui reproduit le comportement attendu est écrit et échoue, puis passe ». Un ticket « expand », qui ajoute sans rien changer à l'écran : elle porte sur les nouveaux tests, et sur les anciens qui restent verts.
- La condition peut faire jusqu'à 4 000 caractères : il y a de la place pour être précis.

À la répétition, sur la 07 de Tailspin (15/10), la condition du premier ticket, écrite à partir de ses critères d'acceptation :

```
/goal Le ticket .scratch/filtrer-les-jeux/issues/01-une-categorie.md
est livré : sur l'accueil, un formulaire filtre le Catalogue par une
Catégorie, de bout en bout. Vérification, visible dans ta sortie :
npm run test:unit, npm run lint, npm run typecheck et npm run build
passent ; le nouveau fichier de test de la famille Catégorie prouve,
via getCatalog, le Catalogue entier sans paramètre, le filtrage par
une Catégorie et les valeurs invalides ; aucun test existant n'est
modifié, ce que montre git diff sur les fichiers de test existants.
Arrête-toi après 10 tours.
```

Atteinte en un tour, en 3 minutes environ : 55 tests, dont 16 nouveaux. Mais la raison de l'évaluateur se termine par une réserve : la vérification au clavier que demandait le ticket n'a pas été faite, et la condition ne l'exigeait pas. Sur le ticket 03 du backlog (les descriptions sur la fiche d'un jeu), il a validé de même, en notant « ...but the condition does not require that rendering check ». La condition, c'est ce que vous avez écrit, pas ce que vous vouliez dire.

Les 5 garde-fous d'une loop fiable

Garde-fou	En pratique	D'après
1. Une condition d'arrêt mesurable, vérifiable dans la sortie	« 0 test en échec », pas « le code est propre »	Anthropic ; doc `/goal`
2. Un plafond de tours	« sinon arrête-toi après 15 tours » ; `--max-iterations` pour `ralph-loop`	Doc `/goal` ; `ralph-loop`
3. Les tests comme contre-pression, et interdits de modification	« aucun fichier `*.spec.ts` n'est modifié »	Huntley ; doc `/goal`
4. Un budget, et un run pilote	Un plafond de dépense (en non interactif, `--max-budget-usd`) ; le statut de `/goal` affiche les tokens, sans plafonner ; un module avant tout le repo	Willison (budget) ; Anthropic, billet sur les loops (run pilote)
5. Un état persistant	Un fichier de progression et des commits réguliers, pour reprendre après une interruption ou un `/clear` (qui efface aussi le goal)	Anthropic ; Huntley

Cités sans être comptés : le sandbox (Willison), dernière couche de la défense en profondeur du J1, et le relecteur à contexte frais : l'évaluateur de `/goal` n'est pas le modèle qui travaille, mais il lit toute sa conversation ; pour un vrai contexte frais, un sous-agent qui relit. Pour votre ticket du jour, l'état persistant peut être très simple : un fichier de progression à côté de la spec, mis à jour à chaque ticket fermé, et un commit par ticket.

Les autres loops

Loop	Comment	Ce qui l'arrête
`/loop` (time-based)	`/loop 5m vérifie si le déploiement est terminé`. Un skill intégré (alias `/proactive`) qui relance le prompt à intervalle, tant que la session reste ouverte. Sans intervalle, Claude choisit son rythme ; sans prompt, il prend un prompt de maintenance intégré, ou votre `.claude/loop.md`.	Vous, ou Claude quand il juge le travail fini ; la fin de la session ; 7 jours au plus
`ralph-loop` (plugin officiel)	`/ralph-loop "<prompt>" --max-iterations 20 --completion-promise "FINI"` : le même prompt, relancé par un hook Stop tant que Claude n'a pas écrit la phrase de fin. Installation : `/plugin install ralph-loop@claude-plugins-official`.	La phrase de fin ; `--max-iterations`, illimité par défaut : fixez-le toujours ; `/cancel-ralph`
Routines (proactive)	Les Routines (cloud), les tâches planifiées du desktop, GitHub Actions : sur un planning ou un événement, sans vous.	Chaque tâche, quand son but est atteint ; la routine, quand vous la coupez

- Ralph vient de Geoffrey Huntley (juillet 2025) : une boucle bash qui relance le même prompt, une tâche par itération, avec les specs et le plan dans des fichiers, et les tests et le typage comme contre-pression. Son auteur le déconseille sur une base de code existante.
- Différence avec `/goal` : avec `ralph-loop`, c'est Claude qui écrit la phrase de fin, donc le modèle qui travaille qui décide qu'il a fini.
- Une surveillance qui doit durer : une Routine plutôt qu'un `/loop`, qui disparaît avec la session.

J2 après-midi — Récap

Scaler par la parallélisation. Le matin, la conception est restée avec vous ; l'implémentation d'un ticket bien spécifié et testé peut passer on the loop, sous le contrôle des tests et de `/goal`. Un développeur qui conçoit peut donc alimenter plusieurs agents… mais tout ce qu'ils produisent, il faut le relire. Le goulot se déplace : on ne manque plus de bras pour écrire le code, on manque d'yeux pour le relire. On parallélise ce qui se relit vite. Puis une dernière brique, le MCP : brancher Claude sur un outil.

Ce que coûte la parallélisation

Chiffre	Ce qu'il mesure	Source
× 4	Les tokens d'un agent, comparé à une conversation de chat	Anthropic, système de recherche multi-agents, juin 2025
× 15	Les tokens d'un système multi-agents, comparé à un chat	Anthropic, même billet
× 2	Le prix au token d'Opus 5.5, comparé à Sonnet 5.5	Tarifs de l'API Anthropic, octobre 2026
+ 441 %	La durée médiane de l'étape de revue des PR (d'après Jira ou Azure DevOps), quand une équipe passe de sa plus faible à sa plus forte adoption de l'IA (une corrélation, pas une causalité)	Faros AI, AI Engineering Report 2026, avril 2026

Le même billet tempère : « most coding tasks involve fewer truly parallelizable tasks than research ». La plupart des tâches de code se parallélisent moins bien que la recherche, parce que leurs morceaux dépendent les uns des autres. Chaque session et chaque sous-agent dépensent leurs propres tokens. Le repère : commencez à deux sessions en parallèle, et n'en ajoutez que si votre relecture et votre attention suivent.

Le coût humain

Paralléliser ne supprime pas le travail humain : ça le déplace vers la vérification. Addy Osmani (« Software Factories, Light and Dark », juillet 2026) : « Verification, not generation, is the real constraint on a factory. » On ne donne à une boucle que l'autonomie qu'on peut vérifier vite et de façon fiable ; et l'humain ne quitte pas l'usine, il change de poste, de la fabrication au réglage de la boucle.

Ce qui disparaît	Ce qui reste
Surveiller chaque étape de l'agent, quand les tests, `/goal` et un relecteur en contexte frais vérifient à votre place	Lire le diff avant qu'il parte, le comprendre, et en répondre en production

Dex Horthy l'a appris à ses dépens (« Why Software Factories Fail », juillet 2026). En juillet 2025, son équipe passe en « usine lights-off » : on lit les specs et les tickets, des agents en arrière-plan font les petites et moyennes tâches, plus personne ne lit le code. En novembre, elle juge plus simple de tout réécrire from scratch. Ses conseils : relire le code, et viser deux à trois fois plus vite, en sécurité, plutôt que dix à cent fois. Faros AI a mesuré le même goulot sur 4 000 équipes : le temps de relecture des PR bondit quand l'adoption de l'IA monte (tableau ci-dessus). La fatigue de la supervision, l'autre face de ce coût, est dans « Quand ça paie, quand s'en passer ».

Worktrees : 1 worktree = 1 agent = 1 tâche

Deux sessions, un seul dossier : non

Lancer plusieurs sessions `/implement` côte à côte dans le même checkout, c'est selon Matt Pocock « worse than unsupported ». Il cite le retour d'un utilisateur qui a vu, en un seul après-midi :

- un `git commit --amend` lancé par une session atterrit sur le commit d'une autre ;
- un stash disparaît ;
- des commits partent sur la mauvaise branche.

La cause : les sessions partagent le dossier de travail (les fichiers sur le disque), l'index (ce qui est prêt à être committé) et le HEAD (la branche courante). Quand l'agent 1 change de branche, il change aussi celle de l'agent 2. Aucun des deux n'a fait d'erreur : c'est l'environnement qui est partagé.

Un worktree git (depuis git 2.5, en 2015) est un dossier de travail séparé, avec ses fichiers, son index et sa branche, qui partage l'historique et le remote du checkout principal. Même historique, autre dossier, autre branche :

Dossier	Branche	Qui y travaille
Le checkout principal	`main`	Vous
`.claude/worktrees/export-csv/`	`worktree-export-csv`	Agent 1
`.claude/worktrees/pagination/`	`worktree-pagination`	Agent 2

Un commit fait dans un worktree arrive dans l'historique commun. Une exception : le stash reste partagé entre worktrees. Dans des sessions parallèles, committez sur votre branche plutôt que de stasher.

En natif dans Claude Code

```
claude --worktree export-csv     # ou : claude -w export-csv
#   dossier : .claude/worktrees/export-csv/
#   branche : worktree-export-csv
claude -w pagination             # 2e terminal : 2e agent, 2e branche

claude --worktree "#1234"        # part de la PR 1234
claude --worktree export-csv --resume   # y revenir plus tard
```

- Sans nom, Claude en génère un. En interactif, `claude -w` exige que vous ayez accepté la confiance du dossier (le dialogue de la première ouverture).
- `--worktree "#1234"` part d'une pull request, ou d'une merge request GitLab : les guillemets sont obligatoires.
- En cours de session, demandez « travaille dans un worktree » : Claude utilise l'outil `EnterWorktree` (et `ExitWorktree` pour en sortir). Pendant qu'une session est isolée dans un worktree, Claude Code bloque les éditions et les commandes git qui viseraient le checkout principal.
- À la sortie d'une session interactive : si le worktree est propre et que la session n'a pas de nom, il est supprimé avec sa branche ; s'il y a du travail dedans, Claude demande s'il faut le garder ou le supprimer (gardez-le si sa branche n'est pas fusionnée). En non interactif (`claude -p`), il n'est jamais nettoyé : `git worktree list` montre ceux qui traînent, `git worktree remove <dossier>` en supprime un. Claude Code laisse en place le verrou posé à la création : si git refuse, `git worktree unlock <dossier>` d'abord.
- Ajoutez `.claude/worktrees/` au `.gitignore`, c'est le conseil de la documentation. Bénéfriches en donne la raison : un dossier de worktrees ignoré par git ne pollue ni `git status`, ni les diffs, ni les recherches.
- Claude Code ne crée pas de worktree sur un nom de branche de votre choix, du genre `feat/export-csv` : pour ça, `git worktree add -b <branche> <dossier>`, puis `claude` dans le dossier.

- Les ADE vus au J1, à l'étage des surfaces de pilotage (Conductor, Orca…), font la même chose : un worktree par agent, avec une interface pour suivre les sessions et relire les diffs. Commencez avec ce que Claude Code fait déjà (`claude -w`, `.worktreeinclude`, `isolation: worktree`) ; un ADE vient après, quand vous avez plusieurs sessions à suivre au quotidien.

D'où part le worktree

Par défaut, le worktree part de la branche par défaut du remote (réglage `worktree.baseRef`, valeur `"fresh"`), pas de votre branche courante. Avec `"head"`, il part de votre HEAD local. Dans les deux cas, il ne contient que ce qui est committé. Pour partir de votre branche, dans `.claude/settings.local.json` (personnel, non versionné ; si le fichier existe déjà, ajoutez la clé sans écraser le reste) :

```
{ "worktree": { "baseRef": "head" } }
```

Autre voie : `git worktree add -b <branche> <dossier>` part du HEAD courant. C'est le piège de l'Atelier 3 : vos tickets de `.scratch/` ne sont dans le worktree que s'ils sont committés et que le worktree part de votre HEAD. Dans Tailspin, `.claude/settings.json` fixe déjà `"baseRef": "head"`, et `.claude/worktrees/` est dans le `.gitignore`.

Avant de créer un worktree, donc :

1. **Committez** sur votre branche de travail ce qu'il doit voir : au J2, le `.scratch/` de votre ticket, `GLOSSARY.md`, `docs/adr/` s'il existe, et le code de votre loop.
2. **Vérifiez** : `git status` est propre, et `git log -1` montre ce commit.
3. **Créez-le** (`claude -w <ticket> --model sonnet`), et vérifiez qu'il voit vos tickets : `!ls .scratch` dans sa session (le point d'exclamation lance la commande vous-même, sans passer par Claude).

Un détail, avec `"head"` : `claude -w` sur un nom qui existe déjà rouvre le worktree à son ancien commit, et raterait ce que vous avez committé depuis. Un worktree de test, quittez-le propre (`/exit`) : il est supprimé, et le prochain `claude -w` le recrée sur le dernier commit.

Ce que git n'isole pas

	Quoi	Que faire
Isolé	Les fichiers, la branche, l'index	Rien
Absent du checkout neuf	Les dépendances installées ; les fichiers ignorés par git, comme `.env`	Relancer l'installation (`npm ci`, `dotnet restore`), la demander à Claude ou lancer le setup du projet. Un fichier `.worktreeinclude`, écrit avec la syntaxe d'un `.gitignore`, copie les fichiers ignorés dans chaque nouveau worktree.
Partagé par la machine	Les ports, la base locale, les volumes Docker	Un port et un nom de projet Docker par worktree. Sur Tailspin, chaque session lance son serveur de dev sur le port libre suivant (4321, 4322…) ; `npx astro dev stop` l'arrête.

Côté .NET, chaque worktree demande son `dotnet restore`, et un `appsettings.Development.json` ignoré par git se copie avec `.worktreeinclude`. Côté Angular, c'est un `npm ci` par worktree : sur un gros projet, c'est long, pensez-y avant d'ouvrir quatre worktrees. Sur Tailspin, rien à installer : un worktree de `claude -w` est rangé sous `.claude/worktrees/`, dans le checkout principal, et Node remonte jusqu'au `node_modules` de ce checkout. C'est un effet de l'emplacement, pas une isolation : si un ticket ajoute une dépendance, son worktree a besoin de son `npm ci`. Ou bien `{ "worktree": { "symlinkDirectories": ["node_modules"] } }` dans vos settings : chaque nouveau worktree pointe sur le `node_modules` du dépôt principal au lieu de le recopier ; à réserver aux tickets qui ne touchent pas aux dépendances, puisqu'il est partagé. Un hook `WorktreeCreate` remplace entièrement la création par git : `.worktreeinclude` n'est alors plus traité, et le script doit tout copier lui-même.

Bénéfriches l'a scripté

Le skill `/worktree` de Bénéfriches (`model: haiku`, `effort: low` : la tâche est mécanique) lance `scripts/create-worktree.sh <branche>`. Le script crée le worktree sous `trees/<nom>`, un dossier ignoré par git, à partir du HEAD courant, puis appelle `scripts/prime-worktree.sh` :

1. `pnpm install` ;
2. le build du package `shared` (sans lui, six fichiers de tests du package échouent) ;
3. la copie des `.env.example` en `.env` ;
4. le tout journalisé dans `.claude-worktree-setup.log`.

Le script d'amorçage sort toujours en 0 : un échec est journalisé sans bloquer, parce que tous les worktrees n'ont pas besoin d'un environnement complet (celui d'un sous-agent qui ne fait que lire, par exemple). Le skill `/worktree-merge` fait le chemin retour. Le dépôt contient aussi un hook prévu pour `WorktreeCreate`, mais il n'est pas déclaré dans le `.claude/settings.json` committé : au commit des démos (`bec2057`), il ne se déclenche pas, et un `claude -w` sur Bénéfriches donne un worktree nu.

Créer un worktree, c'est une commande ; le rendre utilisable, c'est un script à versionner.

Quand ça paie, quand s'en passer

Ça paie	S'en passer
Des tickets indépendants : des tranches verticales	Une tâche de quelques minutes, si préparer le worktree prend plus de temps que la tâche elle-même
Un agent long en arrière-plan (une loop `/goal`, une migration de tests, un audit), pendant que vous travaillez dans le checkout principal	Deux tickets qui touchent les mêmes fichiers ou le même schéma de base : le worktree isole pendant le travail, pas au merge
Comparer deux approches du même ticket : vous gardez la meilleure branche, vous supprimez l'autre	Un refactoring mécanique large : l'expand d'abord, seul
Essayer un refactoring risqué : s'il échoue, on supprime le worktree et sa branche	

La limite, c'est votre relecture et votre attention : commencez à deux. Superviser des agents fatigue : dans une enquête du BCG publiée par la Harvard Business Review (mars 2026, 1 488 salariés), 14 % de ceux qui utilisent l'IA au travail disent avoir vécu cette fatigue mentale, la supervision est la forme d'usage la plus éprouvante, et la productivité ressentie baisse au-delà de trois outils utilisés en même temps. Simon Willison, vingt-cinq ans de métier, raconte qu'avec quatre agents en parallèle il est épuisé vers 11 heures. Exemple .NET : deux tickets qui ajoutent chacun une colonne à la même entité, avec leur migration Entity Framework, produisent en parallèle deux migrations en conflit au merge. Faites-les en séquence, ou dans un seul ticket.

Sous-agents : un contexte frais, un résumé en retour

Un sous-agent est une autre instance de Claude, lancée par l'agent principal. Il travaille dans sa propre fenêtre de contexte, avec son prompt, ses outils et ses permissions, et rend un résultat. L'outil qui le lance s'appelle Agent (il s'appelait Task avant la version 2.1.63 : vous croiserez encore l'ancien nom).

Ce qu'il voit

Il reçoit	Il ne reçoit pas	Il rend
Le message de délégation que Claude lui écrit ; les CLAUDE.md de tous les niveaux, y compris le vôtre ; un instantané de `git status`	Votre conversation : tout ce que vous avez dit à Claude, sauf ce que Claude a recopié dans le message	Un résumé ; ses lectures restent dans sa fenêtre

- Explore et Plan sautent même les CLAUDE.md du démarrage et le `git status`, pour aller plus vite : un Explore ne connaît pas vos conventions générales, sauf si le message les lui donne. Ceux d'un sous-dossier (CLAUDE.md ou AGENTS.md) et les règles à `paths` lui arrivent quand il y lit un fichier.
- La valeur du sous-agent, c'est tout ce qu'il garde hors de votre contexte : des dizaines de fichiers lus, une suite de tests, des centaines de lignes de logs. Votre conversation ne reçoit que la conclusion. C'est la réponse au context rot du J1 : on ne vide pas la fenêtre après coup, on évite de la remplir.
- Le message de délégation est tout ce qu'il saura. Anthropic demande quatre choses pour chaque sous-agent : un objectif, un format de sortie, des consignes sur les outils et les sources, des limites claires.
- Quand les utiliser, d'après la documentation : une tâche dont la sortie est volumineuse et ne vous sert pas telle quelle (une recherche, des logs, des tests) ; un besoin de restreindre les outils ; un travail autonome qui peut se résumer. Restez dans la conversation principale pour les allers-retours, le contexte partagé entre phases et les petits changements.

Intégré (les principaux)	Ce qu'il fait
Explore	Lecture seule, rapide
Plan	Lecture seule, en Plan mode
general-purpose	Tous les outils

Un sous-agent « fork », lui, hérite de toute la conversation : `/subtask <tâche>` en lance un, et en session interactive Claude peut aussi en lancer un de lui-même.

Un sous-agent personnalisé

Un fichier Markdown : `.claude/agents/<nom>.md` pour le projet (versionné avec le code), ou `~/.claude/agents/` pour vous. Seuls `name` et `description` sont obligatoires ; viennent ensuite `tools`, `disallowedTools`, `model`, `permissionMode`, `maxTurns`, `skills`, `mcpServers`, `hooks`, `memory`, `background`, `effort`, `isolation`. L'exemple de Bénéfriches, `.claude/agents/security-reviewer.md` (en-tête remis en forme) :

```
---
name: security-reviewer
description: Proactive security vulnerability
  scanner. Use AUTOMATICALLY after writing
  code that handles user input, …
model: opus
tools: [Bash, Grep, Glob, Read]
---
```

Champ	Ce qu'il fait
`description`	Comme pour un skill, c'est elle qui fait décider Claude. Ici, elle lui demande de l'utiliser automatiquement après du code qui touche des entrées utilisateur, l'authentification, des endpoints, des requêtes SQL ou des données sensibles.
`model`	`opus` : un modèle plus puissant pour une relecture de sécurité ; juger, c'est le travail d'Opus (voir « Un modèle par phase »). Sans ce champ, le sous-agent hérite du modèle de la session.
`tools`	Restreint : c'est la liste des outils autorisés, ici ni Edit ni Write. Dans un skill, `allowed-tools` pré-approuve et ne restreint rien : même allure, effet opposé. Et Bash reste dans la liste : « pas d'Edit » ne veut pas dire « ne peut rien écrire ». Sans `Agent` dans la liste, il ne peut pas lancer de sous-agent : la garantie contre la cascade (voir « Ce qui casse »).
`isolation: worktree`	Un worktree temporaire à chaque exécution, supprimé s'il n'a rien modifié : un sous-agent qui écrit du code ne touche pas à votre dossier. Même piège que pour `claude -w` : ce worktree part de la branche par défaut du remote, sauf avec `worktree.baseRef: "head"` ; sans ce réglage, le sous-agent ne voit ni vos commits locaux ni `.scratch/`.
`mcpServers`	Ses serveurs MCP : un serveur déjà configuré, désigné par son nom, ou une définition en ligne. Déclaré en ligne, le serveur n'existe que pour lui, pas pour la conversation principale (voir « Le confier à un sous-agent », dans la partie MCP).
`skills`	Injecte le contenu de ces skills à son démarrage : par exemple, un relecteur qui démarre avec votre skill de revue du J1. Inversement, dans un skill, `context: fork` le fait tourner dans un sous-agent.

Idées pour votre stack : en .NET, un relecteur de migrations Entity Framework en lecture seule (`tools` : Read, Grep, Glob), « à utiliser après chaque nouvelle migration » ; en Angular, un relecteur d'accessibilité des templates.

Le cycle de vie

1. **Lancé** par Claude quand sa `description` correspond à la tâche, ou par vous : « utilise un sous-agent pour… », ou en le nommant.
2. **En arrière-plan** par défaut, en session interactive : vous pouvez continuer à parler à Claude. `/tasks` liste ce qui tourne ; Ctrl+B passe une tâche en arrière-plan.
3. **Le résultat** revient comme une notification, dans un tour suivant.
4. **Relancé** avec SendMessage, un sous-agent nommé garde tout son historique : utile pour lui renvoyer des corrections sans tout lui réexpliquer.

Les limites. Par défaut, 20 sous-agents en même temps au plus, et un sous-agent peut en lancer d'autres jusqu'à 3 niveaux sous la conversation principale : une possibilité, pas un conseil (voir « Ce qui casse »). Chaque sous-agent dépense ses propres tokens : plusieurs sessions ou sous-agents en même temps, c'est une consommation de tokens multipliée d'autant (documentation, page « Run agents in parallel »).

À ne pas confondre	Ce que c'est
Les agent teams	Expérimentales, désactivées par défaut (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`) : un chef d'équipe et des coéquipiers qui se parlent directement, sans worktrees (ils partagent aussi une liste de tâches quand les outils de tâches sont là, ce qui n'est pas le cas par défaut sur Opus 5.5 et Sonnet 5.5). Un sous-agent, lui, ne rapporte qu'à la conversation qui l'a lancé.
`/batch <instruction>`	Un skill intégré : il découpe un gros changement en 5 à 30 unités indépendantes, vous présente un plan, puis lance un sous-agent par unité, chacun dans son worktree, qui implémente, teste et publie son changement.

Un modèle par phase

Opus juge, Sonnet exécute. La documentation de Claude Code (page « costs ») : Sonnet fait bien la plupart des tâches de code et coûte moins cher ; Opus est à garder pour les décisions d'architecture complexes et les raisonnements en plusieurs étapes. Mettre la conception et la relecture sur Opus en est une application, et un choix du support. Au prix de l'API, Opus 5.5 coûte deux fois Sonnet 5.5 au token, et il use les limites d'usage plus vite.

Phase	Modèle	Comment
Concevoir : `/grill-with-docs`, `/to-spec`, `/to-tickets`	Opus, le défaut sur Pro, Max, Team et Enterprise	`/model opus` pour le vérifier
Implémenter un ticket spécifié et testé	Sonnet	`claude -w <ticket> --model sonnet` : pour cette session seulement
Relire, juger	Opus	« un sous-agent Opus » dans le prompt, ou `model: opus` dans son fichier
Une tâche étroite ou mécanique	Haiku	`model: haiku` dans son fichier : le skill `/worktree` de Bénéfriches, le sous-agent concierge de l'exercice MCP

- `--model` ne vaut que pour la session lancée ; `/model` change tout de suite, mais devient aussi le défaut de vos sessions suivantes. Un `/model sonnet` tapé pour implémenter : repassez sur Opus avant de concevoir le ticket suivant.
- Le modèle d'un sous-agent : celui que Claude demande à l'appel (c'est ce que produit « un sous-agent Opus » dans un prompt) l'emporte sur le champ `model` de son fichier ; sans précision, le sous-agent hérite du modèle de la session.
- Les sous-agents Explore suivent eux aussi le modèle de la session : pour une analyse parallèle, lancez-la en Sonnet (`claude --model sonnet`) ; trois Explore sur Opus useraient vos limites d'usage pour une tâche de lecture.
- Les skills de Matt Pocock ne fixent aucun modèle : `/implement` et les relecteurs de son `/code-review` tournent sur le modèle de la session.

Fan-out et pipeline : deux façons d'assembler

	Fan-out	Pipeline
Le principe	L'orchestrateur (la conversation principale) lance plusieurs sous-agents en même temps sur des tâches indépendantes, souvent en lecture	Des étapes qui s'enchaînent, chacune dans un contexte frais : implémenter, tester, relire, committer
Ce qui circule	Un rapport par sous-agent (par exemple `tests.md`, `observabilite.md`, `performance.md`), puis une synthèse (`roadmap.md`)	Un diff, des résultats de tests, des constats
Le goulot	La synthèse : des doublons, parfois des contradictions, à fusionner et à vérifier	Le passage de relais : « j'ai fini, tout marche » ne sert à rien à l'étape suivante

Le fan-out, c'est l'architecture orchestrateur-workers décrite par Anthropic. Le pipeline du programme enchaîne implémenter, tester, relire, committer ; celui du prompt ci-dessous le resserre : les tests dans l'implémentation, une correction, et le commit, c'est vous. La chaîne du matin (`/grill-with-docs` → `/to-spec` → `/to-tickets` → `/implement`) est déjà un pipeline : la conception reste dans votre conversation, et chaque implémentation peut tourner dans un contexte frais. Dans les deux cas, un fichier sur chaque flèche : un fichier se relit, se versionne, survit à un `/clear`, et c'est tout ce que le contexte frais de l'étape suivante a besoin de lire.

Le prompt du pipeline

Implémenter, relire, corriger, en un prompt, sur un ticket produit le matin par `/to-tickets` :

```
Prends le ticket .scratch/<fonctionnalité>/issues/01-….md.
Attends le rapport de chaque sous-agent avant l'étape suivante.
1. Implémentation : un sous-agent Sonnet nommé impl. Il ne lance
   pas d'agent. Il lance les tests ciblés avant son rapport ;
   un critère d'acceptation = une preuve.
2. Revue : un sous-agent Opus, en lecture seule, relit le diff
   contre le ticket, sans le rapport d'impl. Il ne corrige rien
   et ne lance pas d'agent. Il ne signale que ce qui touche la
   correction ou le ticket : bloquant / important / mineur,
   avec fichier:ligne.
3. Correction : renvoie les constats à impl avec SendMessage,
   un tour au plus.
4. Ne committe pas. Termine par : fait / reste à faire.
```

Chaque ligne répond à un piège :

Dans le prompt	Le piège évité
« Attends le rapport… »	En session interactive, les sous-agents partent en arrière-plan : sans cette ligne, la revue peut partir avant la fin de l'implémentation, ou Claude vous rend la main en annonçant « c'est lancé ».
Un sous-agent nommé `impl`	C'est lui qu'on relance à l'étape 3, avec son historique : il a déjà le code en tête.
Sonnet pour `impl`, Opus pour la revue	Le modèle fort là où on juge, le modèle moins cher là où on exécute un ticket déjà spécifié. Claude passe le modèle à l'outil Agent.
« Il ne lance pas d'agent »	La cascade : un sous-agent qui lance ses propres sous-agents (voir « Ce qui casse »).
Les tests ciblés ; un critère d'acceptation = une preuve	Pas toute la suite. Chaque case à cocher du ticket est prouvée par un test qui passe ou une sortie de commande.
Le relecteur ne reçoit pas le rapport d'`impl`	Il juge sur le ticket et le diff, pas sur ce que l'autre affirme. La doc : « Claude won't be biased toward code it just wrote ».
Il ne signale que ce qui touche la correction ou le ticket	Un relecteur à qui on demande des trous en trouve presque toujours.
Il ne corrige rien	Sinon, plus personne ne sait qui a écrit quoi.
Un tour au plus	Sinon, les deux peuvent se renvoyer la balle sans fin.
Ne committe pas	La dernière relecture, c'est la vôtre ; « fait / reste à faire » est le rapport que vous lisez.

Un piège hors du prompt : si la variable `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` est à 1 chez vous, tout sous-agent que Claude nomme depuis la conversation principale part en coéquipier (sauf un fork, ou un appel avec `isolation`), et le pipeline devient une agent team. Remettez-la à 0, et relancez la session.

Prêts à l'emploi : `/implement` de Matt Pocock fait la même chose pour un ticket. Il pilote `/tdd`, puis son `/code-review`, qui lance deux relectures en sous-agents parallèles : Standards (le code suit-il les règles du repo ?) et Spec (fait-il ce que demandait le ticket ?). Un bug connu, signalé dans sa documentation (issue #511) : ce `/code-review` ne relit que ce qui est committé, et `/implement` le lance avant de committer ; sans commit intermédiaire, la revue ne voit rien du ticket. Demandez un commit avant la revue, ou relancez-la vous-même après le commit : dans le menu, elle s'appelle `/mattpocock-skills:code-review`. Claude Code a aussi son skill

intégré `/code-review`, qui cherche les bugs dans le diff : même nom court, autre outil. Les skills de Pocock ne fixent aucun modèle : dans une session Sonnet, ses relecteurs tournent aussi sur Sonnet ; pour un juge Opus, prenez le prompt ci-dessus.

Ce qui casse

Piège	Ce qui se passe	Parade
Des rôles : un agent front, un agent back, un agent QA	Un découpage par personnage, sans rien gagner en contexte ; un agent front et un agent back, c'est le découpage par couche	Découper par contexte : un sous-agent garde hors de votre fenêtre une sortie volumineuse, ou porte un regard frais
La cascade	Un sous-agent lance les siens, jusqu'à 3 niveaux par défaut : les tokens se multiplient, plus personne ne sait qui a fait quoi	« Il ne lance pas d'agent » dans le brief ; pour une garantie, un sous-agent défini sans `Agent` dans ses `tools`, ou `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH=1`, qui coupe l'imbrication
Le rapport qui ment	Un sous-agent affirme ce qu'il n'a pas vérifié	Un chemin par affirmation, NON VÉRIFIÉ pour le reste ; vous vérifiez vous-même une affirmation
Le coût	Chaque sous-agent paie ses tokens, et son rapport entre dans votre fenêtre	Moins de sous-agents, des rapports courts écrits dans des fichiers

Dex Horthy (« No Vibes Allowed », AI Engineer Code Summit, novembre 2025) : « Sub-agents are not for anthropomorphizing roles, they are for controlling context. » Un prompt de rôle n'est pas un problème en soi (la documentation de Claude Code écrit elle-même « You are a senior code reviewer ») ; le piège, c'est de découper le travail par personnage.

La cascade existe pour de vrai : les deux relecteurs du `/code-review` de Matt Pocock, que lance `/implement`, peuvent relancer `/code-review` eux-mêmes. La page du skill cite un signalement à plus de cinquante agents, reproduit dans plusieurs harness (issue #573 de `mattpocock/skills`, juillet 2026). Pocock a fermé l'issue sans correctif en octobre : pour lui, c'est au harness d'arrêter ces boucles, pas aux skills. Dans Claude Code, la limite de trois niveaux arrête la cascade à quatorze relecteurs (2, puis 4, puis 8) : c'est borné, mais c'est sept fois le prix d'une relecture. Si la liste de `/tasks` s'allonge : `Ctrl+X Ctrl+K`, deux fois en trois secondes, arrête les sous-agents en arrière-plan (Échap seul ne les arrête pas).

Un harness qui fait son travail, vu à la préparation sur Tailspin : le pipeline a trouvé que le ticket 02 cassait des tests du 01, qu'il n'avait pas le droit de toucher. Il n'a pas levé l'interdit lui-même : il a rendu la main avec deux options. C'est ce qu'on attend d'un harness : il remonte ce qu'il ne peut pas trancher.

Découper : par feature, couche, domaine ou étape

Découpage	Pour quoi faire	Avec quoi
Par feature, en tranche verticale	Construire en parallèle	Un worktree par ticket
Par couche : front, API, base	À éviter pour paralléliser	—
Par domaine : sécurité, tests, observabilité, performance	Analyser en parallèle	Des sous-agents en lecture seule
Par étape	Enchaîner	Un pipeline de sous-agents

Par couche, chaque ticket attend les autres pour être testable : le front ne fonctionne pas sans l'API, l'API pas sans la base. Aucun ne se vérifie seul, donc aucun agent ne peut savoir qu'il a fini. La documentation de `/to-tickets` rapporte le cas d'une équipe : 26 tickets découpés par couche, environ 20 passages d'agent par

ticket fermé, dont les trois quarts à refaire ; leur post-mortem attribue tous les échecs au découpage horizontal.

Exemple, l'export CSV de la liste des commandes en Angular. Par couche : un ticket pour le service, un pour le composant, un pour le test e2e ; aucun n'est démontrable seul. En tranches : 01, un bouton qui exporte la liste en CSV (service, composant et test) ; 02, l'export avec le filtre courant ; 03, les colonnes du tableau ; 04, l'export complet, filtre et colonnes ensemble, vérifié de bout en bout. Chaque tranche se montre à un utilisateur.

Le graphe décide

Ticket	Bloqué par	Quand il part
01 Export minimal : bouton → fichier CSV	Rien	Tout de suite : un agent
02 Avec le filtre courant	01	Quand 01 est fermé, en parallèle de 03 : un worktree chacun
03 Colonnes du tableau	01	Quand 01 est fermé, en parallèle de 02
04 Export complet, vérifié de bout en bout	02, 03	Quand les deux sont fermés

La frontière, ce sont les tickets dont tous les bloqueurs sont fermés. Sur ce graphe, jamais plus de deux agents utiles, et seulement au milieu. La 07 de Tailspin a la même forme : 01, une catégorie de bout en bout ; puis 02, l'éditeur, et 03, plusieurs catégories, en parallèle ; puis 04, la combinaison. Sur Bénéfriches, à la préparation du support, le graphe était linéaire (01 ajouter une fonction explicite en m², 02 y passer les écrans de saisie, 03 basculer `formatSurfaceArea` en hectares, 04 vérifier les e2e et le PDF) : un expand puis une migration, un agent à la fois, et on le savait avant d'en lancer quatre. Le graphe dépend aussi de l'état du code : à une première répétition, où une fonction de formatage existait déjà, le découpage donnait 01, puis 02, 03 et 04 en parallèle.

Les trois questions de la documentation (page « Run agents in parallel ») :

Question	Réponse
Qui coordonne ?	Claude, dans une conversation : des sous-agents. Vous, qui confiez des tâches indépendantes et repassez plus tard : des sessions, et la vue agents (`claude agents`, en research preview). Claude qui planifie et supervise une équipe : une agent team (expérimental). Un script qui tient le plan à la place de Claude : les workflows dynamiques (`/workflows` pour les suivre). (La boucle shell `claude -p` vient de la page Best practices.)
Les agents doivent-ils se parler ?	Les sous-agents rapportent à la conversation qui les a lancés (des sous-agents nommés peuvent aussi s'écrire). Les sessions que vous lancez vous-même peuvent se passer leurs résultats par la messagerie entre sessions. Les coéquipiers d'une agent team se parlent directement.
Touchent-ils les mêmes fichiers ?	Oui : un worktree chacun.

Ce n'est pas vous qui décidez combien d'agents : c'est le graphe, et votre relecture. Pour l'Atelier 3 : vos tickets de la frontière écrivent du code, c'est vous qui coordonnez, et ils n'ont pas à se parler. Donc une session et un worktree par ticket. La version 1.3.1 des skills de Pocock a un skill `/implement-spec` qui automatise exactement ça : les tickets de la frontière partent chacun dans un sous-agent, dans son propre worktree.

Le prompt d'analyse parallèle

Un fan-out en lecture : trois sous-agents qui auditent le même module, chacun sur son axe, puis une synthèse. C'est l'étape 4 de l'Atelier 3, et son repli pour un projet sans découpage ni tests. Le prompt, en modèle à adapter : `<module>` est le dossier à auditer, `<axe 1>` à `<axe 3>` vos axes, `<nom>` un nom court pour le dossier des rapports.

```
Audite le module <module> sur trois axes, en parallèle : <axe 1>, <axe 2>, <axe 3>. Lance trois sous-
agents Explore (lecture seule), un par axe, en même temps.

Brief de chaque sous-agent :
- Objectif : les 3 problèmes les plus importants sur ton axe, classés par impact.
- Sortie : pour chaque problème, le constat, l'impact, fichier:ligne et la correction proposée.
- Limites : reste dans <module> et sur ton axe ; ne modifie rien ; ne lance pas d'agent.
- Cite un chemin pour chaque affirmation ; marque NON VÉRIFIÉ ce que tu n'as pas vérifié.

Attends les trois rapports. Écris-les dans .scratch/audit-<nom>/<axe>.md, puis fais la synthèse dans
.scratch/audit-<nom>/roadmap.md : doublons fusionnés, les trois premières actions. Vérifie toi-même
une affirmation de chaque rapport en ouvrant le fichier cité, et dis laquelle.
```

- Sur Tailspin : le module `src`, les axes tests, accessibilité et sécurité, le nom `src` ; les rapports arrivent dans `.scratch/audit-src/`.
- Axes possibles : tests, observabilité, performance, sécurité, accessibilité. Pour un repo .NET, par exemple : tests, observabilité, performance des requêtes, sécurité. Avec quatre axes, écrivez « quatre » partout où le prompt dit « trois ».
- Le brief reprend les quatre points d'Anthropic : un objectif, un format de sortie, des consignes sur les sources (« cite un chemin… »), des limites.
- Le rapport d'un sous-agent est une hypothèse, pas une preuve (Pocock) : d'où la vérification d'une affirmation par rapport, par l'orchestrateur ; puis la vôtre, sur une affirmation de `roadmap.md` qu'il n'a pas vérifiée, en ouvrant le fichier cité.
- Lancez la session en Sonnet (`claude --model sonnet`) : les sous-agents Explore suivent le modèle de la session. Restez en mode Auto, ou passez en Accept edits : l'orchestrateur écrit quatre fichiers, sans quatre demandes de permission.
- Fusionnez d'abord ce qui doit être audité : l'analyse porte alors sur le code complet, et rien ne change sous les sous-agents pendant qu'ils lisent.

Ce qu'a donné la répétition sur Tailspin (15/10, Sonnet 5.5, mode Auto, en non interactif), en 1 min 40 s : les trois sous-agents Explore sont partis dans le même tour ; l'orchestrateur a annoncé qu'il attendait, puis compté les rapports qui arrivaient. La roadmap fusionnait un doublon, la route `src/pages/api/games/[id].ts`, vue par les rapports sécurité et tests : un id validé par une regex plutôt que par la fonction de validation du projet, pas de `try/catch`, l'id renvoyé dans le message 404. Elle proposait trois actions : corriger et tester cette route ; l'accessibilité (`about.astro` sans `h1`, le même `<title>` sur toutes les fiches, `StarRating` muet pour un lecteur d'écran) ; les tests de `src/lib/db.ts`. Une affirmation vérifiée par rapport, et des NON VÉRIFIÉ honnêtes : les contrastes, le rendu du focus, l'adaptateur Node en production.

L'Atelier 3, geste par geste

Les cinq étapes de l'atelier, à refaire au calme sur un ticket de votre mission ; la consigne détaillée est dans votre cahier. C'est le livrable 4 du programme : un workflow multi-agent testé avec worktrees et sous-agents. Pour le refaire sur Tailspin, le découpage de secours de la 07 est sur les branches du fork : `git fetch origin`, puis `git switch secours/07-filtres` (la spec et les quatre tickets), ou `secours/07-filtres-01-fait` (le premier ticket déjà fait).

La règle de tout l'atelier : tant que le harness ne remonte rien, vous n'intervenez pas. Remonter, c'est une demande de permission (vous y répondez), un hook qui bloque, un verdict « impossible » de `/goal`, un plafond de tours atteint, une session qui rend la main avec un « reste à faire ». Le reste du temps, chaque envie de reprendre la main se note, avec sa raison.

1. Deux tickets, deux worktrees, en même temps. Un terminal par ticket de la frontière, à la racine du projet :

```
# Terminal 1 : le skill de Pocock
claude -w <ticket-1> --model sonnet
/implement <chemin du ticket 1>

# Terminal 2 : votre pipeline
claude -w <ticket-2> --model sonnet
<le prompt du pipeline, avec le chemin du ticket 2>
```

Vérifiez « ▸▸ auto mode on » dans chaque session (sinon, Shift+Tab), puis lancez les deux avant d'en regarder un : c'est ça, le parallèle. `/implement` committe ; le pipeline ne committe pas, il termine par « fait / reste à faire ». Un seul ticket sur la frontière, parce que le graphe est linéaire : le pipeline seul, dans un worktree, c'est déjà un workflow multi-agent.

2. Relire, contre le ticket. Dans chaque session :

```
!git show --stat            # /implement : il a committé
!git add -N . && git diff   # le pipeline : rien n'est committé
```

`git diff` seul ignore les fichiers non suivis : sans `git add -N`, qui les fait entrer dans le diff sans les indexer, vous ne verriez ni le nouveau test ni les nouveaux composants. Le ticket ouvert à côté, critère par critère : ce qui le prouve, c'est une ligne du diff ou un test, pas le rapport de l'agent. Ce qu'aucun test ne prouve, vérifiez-le à la main : « Lance le serveur de dev et donne-moi l'URL », un port par session. Un défaut trouvé : renvoyez-le à la session (« Le critère 3 n'est pas prouvé : … »), ne le corrigez pas vous-même dans l'éditeur. Le travail du pipeline que vous gardez : « Committe ce ticket sur cette branche. » Claude ne committe pas de lui-même : sans commit, il n'y a rien à fusionner.

3. Fusionner : la frontière avance. Dans un troisième terminal, à la racine, sur votre branche de travail :

```
git merge --no-edit worktree-<ticket-1>
git merge --no-edit worktree-<ticket-2>
npm run test:unit
```

`--no-edit` évite l'éditeur du message de merge. Un conflit : `claude` dans ce terminal, puis « Résous le conflit, et lance les tests » ; vous relisez, et vous le notez : le découpage n'était pas si indépendant. Le ticket que ces deux-là bloquaient est maintenant sur la frontière : écrivez sa condition `/goal`, puis, dans un quatrième terminal, `claude -w <ticket-3> --model sonnet`, et lancez-la. Le piège : quand les deux fusions suffisent à faire marcher le ticket suivant, « les tests passent » est vrai avant le premier tour ; la condition nomme le nouveau test. Vous préférez `/implement` pour ce ticket : marquez d'abord ses bloqueurs comme fermés (leur ligne `Status:` dans `.scratch/`), committés sur la branche de travail, puis `claude -w`.

4. Analyser en parallèle. Le verdict de la loop, dans sa session : `/goal`, puis Ctrl+O. Atteinte : relisez son diff (`!git add -N . && git diff`), faites-la committer, puis fusionnez sa branche dans le terminal 3 et relancez les tests. Pas atteinte : la raison de l'évaluateur va dans la grille ; c'est souvent la condition, pas l'agent, et on ne fusionne pas. Puis, dans le terminal 3, `claude --model sonnet` et le prompt d'analyse parallèle, sur ce code complet.

5. Classer vos tentations. Chaque tentation d'intervenir, rangée dans l'une des quatre classes de « Quand vous rentrez dans la boucle » (J2 matin) : harness, spec, code, ou vraie erreur d'agent.

Ma tentation	Ce qui a cloché	Ce que je corrige
« Le relancer : il a modifié un test »	Le harness : une vérification manquait	« aucun test n'est modifié » dans la condition, ou un hook
« Lui expliquer le format de date attendu »	La spec : elle avait un trou	Une ligne dans la spec, puis relancer

La question à se poser : qu'est-ce qui aurait empêché l'erreur ? S'il y a une réponse, ce n'est pas une erreur d'agent. On se classe trop vite en « erreur d'agent », parce que c'est la seule classe qui ne demande rien de nous. La correction la plus utile rejoint votre plan d'adoption. Puis on range : `/exit` dans chaque session (gardez un worktree dont la branche n'est pas fusionnée), `npx astro dev stop` là où tourne un serveur de dev, et `git worktree list` pour voir ce qui reste.

Le MCP : brancher Claude sur un outil

Le Model Context Protocol est un standard ouvert : un serveur MCP annonce à Claude Code, son client, des outils (des fonctions que le modèle appelle, avec l'accord de l'utilisateur), des ressources (des données à lire) et des prompts (des modèles prêts à l'emploi). Au J2, on n'a fait que des outils : un serveur sur le catalogue de Tailspin, puis le sous-agent qui s'en sert.

Un serveur, de près

Un extrait simplifié du serveur que Claude a écrit à la répétition (`mcp/catalog-server.ts`) :

```
server.registerTool('search_games', {
  description: 'Search the Tailspin Toys catalog. Case-insensitive'
    + ' text matched against title, category and publisher.',
  inputSchema: z.object({ query: z.string().trim().min(1) }),
  annotations: { readOnlyHint: true },
}, async ({ query }) => json(await searchGames(db, query)));

await server.connect(new StdioServerTransport());
```

- `registerTool` déclare un outil : son nom, sa description, son schéma d'entrée (en zod), une annotation (`readOnlyHint` : il ne modifie rien) et la fonction qui répond. Elle appelle `searchGames`, une fonction de `src/lib/` testée par Vitest : le serveur ne fait que l'exposer.
- **Le transport.** stdio : Claude Code lance le serveur comme un processus local, et lui parle par stdin et stdout, en JSON-RPC. HTTP : le serveur tourne ailleurs, on donne une URL ; c'est le cas du plugin Context7 de la marketplace officielle (celui de Bénéfriches, vu au J1, tourne en stdio), du serveur de Jira et de la version distante de celui d'Azure DevOps.
- **La description** décide quand Claude appelle l'outil, comme celle d'un skill ou d'un sous-agent.
- **stdout porte le protocole.** La documentation du protocole le dit en toutes lettres : y écrire corrompt les messages JSON-RPC et casse le serveur. En TypeScript, pas de `console.log` (les diagnostics partent sur stderr, avec `console.error`) ; en C#, pas de `Console.WriteLine` ; en Java, pas de `System.out.println`.
- **Le SDK.** Le SDK officiel en TypeScript est passé en v2 fin juillet 2026, en deux paquets, `@modelcontextprotocol/server` et `@modelcontextprotocol/client` ; la v1, `@modelcontextprotocol/sdk`, ne reçoit plus que des correctifs. Le tutoriel officiel, « Build an MCP server », existe aussi en C#, en Java, en Python, en Kotlin, en Go, en Rust et en Ruby.

Le prompt de l'exercice, à la racine du projet, sur une branche neuve (`git switch -c mcp-catalogue`), dans une session `claude --model sonnet`. Sur votre projet, adaptez-le à une donnée simple (un fichier de données, une table de dev en lecture, une API locale), en lecture seule d'abord :

```
Écris un serveur MCP en stdio pour le catalogue, dans
mcp/catalog-server.ts, avec le SDK officiel v2
(@modelcontextprotocol/server, et zod). Deux outils en lecture
seule, qui réutilisent src/lib/ : search_games (un texte, cherché
dans le titre, la catégorie et l'éditeur) et get_game (par id).
La logique va dans src/lib/, testée par Vitest ; le serveur ne
fait que l'exposer. Ne lance pas d'agent.
```

Nommer le SDK v2 évite que Claude prenne l'ancien ; s'il le prend, ça marche aussi, seuls les imports changent. À la répétition, en 1 min 31 s : une fonction `searchGames` et 11 tests dans `src/lib/`, un serveur de 58 lignes, ses messages d'erreur sur stderr, et le chemin de la base calculé depuis le fichier du serveur, non depuis le dossier courant. Lisez le serveur : la description de chaque outil, et où partent ses messages d'erreur. Ses tests prouvent la recherche, pas le serveur : le serveur, on le teste en s'en servant. Lancé dans un terminal, il reste muet et bloqué : c'est un serveur qui attend un client (Ctrl+C pour l'arrêter).

Le brancher

```
claude mcp add --scope project tailspin -- npx tsx mcp/catalog-server.ts
claude mcp list        # tailspin : ⏸ Pending approval
```

Après `--`, la commande qui lance le serveur. Sans `--transport`, c'est stdio ; pour un serveur distant, `claude mcp add --transport http <nom> <url>`.

Portée	Rangé dans	Pour qui
`local`, par défaut	`~/.claude.json`	Vous, sur ce projet
`project`	`.mcp.json`, à la racine, versionné	L'équipe, après approbation
`user`	`~/.claude.json`	Vous, sur tous vos projets

- Un serveur de `.mcp.json` s'approuve au premier lancement interactif : un `.mcp.json` cloné peut lancer du code sur votre poste, c'est la logique du dialogue de confiance du J1. Le dialogue « New MCP server found in this project » s'ouvre avec le curseur sur « Continue without using this MCP server » : Entrée par réflexe, et le serveur est ignoré. `claude mcp reset-project-choices` remet ces choix à zéro.
- Dans la session, `/mcp` affiche le statut de chaque serveur et ses outils (« ✔ tailspin · 2 tools », puis View tools), et permet de le reconnecter ou de le désactiver.
- Ses outils s'appellent `mcp__<serveur>__<outil>`. C'est ce nom qu'on écrit dans les permissions (`mcp__tailspin__*` autorise tous ses outils ; une règle `mcp__*` seule est ignorée dans `allow`), dans le `tools` d'un sous-agent, ou dans le matcher d'un hook (`mcp__tailspin__.*`) : les hooks du J1 s'appliquent aussi aux outils MCP.
- Le coût : au démarrage, seuls les noms des outils sont chargés ; leurs définitions arrivent à la demande (la section « MCP Tools » de `/context`). Une sortie d'outil est plafonnée à 25 000 tokens (`MAX_MCP_OUTPUT_TOKENS`), avec un avertissement dès 10 000.
- « ✘ Failed to connect » : lancez `claude` à la racine du projet (le chemin du serveur est relatif), ou lancez le serveur à la main pour voir son erreur, puis `/mcp`, Reconnect. Sous Windows, « Connection closed » sur un serveur lancé par `npx` : `claude mcp remove tailspin -s project`, puis le même `claude mcp add`, avec `cmd /c` avant `npx`.
- L'app desktop (onglet Code) embarque sa propre copie de Claude Code, hors du PATH : la commande `claude` n'existe pas dans le terminal. Faites écrire `.mcp.json` par Claude, puis ouvrez une nouvelle session (Ctrl+N), *Local*, sur la racine du projet, sans worktree.

S'en servir : Claude a souvent un chemin plus court

Branché, le serveur ne sert pas forcément. À la répétition, à « Quel est le jeu le mieux noté de chaque catégorie ? », Claude a lu le schéma et interrogé `tailspin.db` lui-même, dans une commande shell : 11 s, un tableau juste, et aucun appel au serveur. Avec « en passant uniquement par les outils du serveur tailspin », il l'a appelé cinq fois, un `search_games` par catégorie : 6 s, le même tableau. Ctrl+O montre par où il est passé. À une question hors catalogue (« Combien a levé Repo Rulers ? »), on attend qu'il dise que le catalogue ne le sait pas : s'il avance un chiffre, c'est une invention.

Dans un repo, un outil MCP n'est qu'un chemin parmi d'autres. Ici, `search_games` cherche un texte : il sert mal « le mieux noté de chaque catégorie », alors que la base est à portée de shell. Deux leviers : un outil taillé pour les questions qu'on posera, avec une description qui les nomme ; ou retirer les autres chemins, en le confiant à un sous-agent.

Le confier à un sous-agent

Déclaré en ligne dans l'en-tête d'un sous-agent, le serveur n'existe que pour lui : la conversation principale n'en voit plus rien. Le prompt de l'exercice :

```
Crée le sous-agent concierge, dans .claude/agents/concierge.md :
il répond aux questions sur le catalogue en suivant les règles de
backlog/09-backer-concierge.md, sur Haiku, avec pour seuls outils
ceux du serveur tailspin. Déclare ce serveur dans son en-tête, en
ligne, avec la même commande que dans .mcp.json, puis retire-le
de .mcp.json : la conversation principale ne doit plus le voir.
Ne lance pas d'agent.
```

L'en-tête écrit par Claude à la répétition (remis en forme) :

```
---
name: concierge
description: Répond aux questions des backers sur le catalogue
  Tailspin Toys (…)
model: haiku
tools: mcp__tailspin__search_games, mcp__tailspin__get_game
mcpServers:
  - tailspin:
      type: stdio
      command: npx
      args: [tsx, mcp/catalog-server.ts]
---
```

- `tools` restreint aux deux outils du serveur : sans ce champ, le sous-agent hérite des outils de la session, Bash compris, et trouve un autre chemin. À la répétition, un concierge sans `tools`, dont le serveur ne s'était pas chargé, a parlé au serveur en JSON-RPC… par Bash.
- Haiku : une tâche étroite, en lecture, sur deux outils.
- Sous Windows, si le branchement a demandé `cmd /c`, l'en-tête du concierge doit le reprendre (`command: cmd`, `args: [/c, npx, tsx, mcp/catalog-server.ts]`) : d'où « avec la même commande que dans .mcp.json » dans le prompt.
- Un dossier `.claude/agents/` créé pendant la session ne se charge qu'au démarrage suivant : `/exit`, puis relancez `claude`. Et un serveur déclaré dans un fichier de `.claude/agents/` ne se charge que dans un dossier de confiance ; sinon, « 'concierge' would be spawned with zero tools — refusing ».
- Le concierge n'est pas appelé : nommez-le, « Demande au concierge : … ».
- À la répétition, à « Demande au concierge : quel jeu de stratégie est le mieux noté, et combien a-t-il levé ? », le concierge, sur Haiku, a appelé trois fois `search_games` puis `get_game`, et rendu en 11 s : Repo Rulers, 4,1, Ops Interactive, et aucun montant levé dans le catalogue. Et `/context` ne montre plus aucun outil tailspin dans la conversation principale.

Quand, et à quel prix

Le besoin	La brique
Une CLI que Claude connaît déjà : `gh`, `az`, `dotnet`	Un skill qui dit comment s'en servir sur votre projet
Un système hors de sa portée, derrière une authentification : Jira, Azure DevOps, une API interne	Un serveur MCP, souvent publié par l'éditeur
Une interface étroite pour un agent : deux outils en lecture, pas de Bash	Votre serveur, confié à un sous-agent

C'est un repère tiré de l'exercice, pas une règle de la documentation. Deux serveurs d'éditeurs, qui n'ont pas été testés pendant la préparation :

- **Jira** : le serveur d'Atlassian (Rovo MCP Server), distant, hébergé par Atlassian, pour Atlassian Cloud seulement ; il couvre Jira, Confluence et Bitbucket entre autres, avec OAuth 2.1 ou un jeton d'API : `claude mcp add --transport http atlassian https://mcp.atlassian.com/v2/mcp`.
- **Azure DevOps** : le serveur officiel de Microsoft, en préversion, local par `npx` ou distant, le distant étant recommandé ; il expose les work items, les repos, les pipelines, les plans de test, les wikis et les builds.

Sur un poste client, brancher un serveur est une décision du client. Trois questions avant de le faire : y a-t-il une CLI que Claude connaît déjà ? un serveur publié par l'éditeur ? qui doit l'autoriser ?

La sécurité. Un serveur stdio est un programme qui tourne sur votre poste, avec vos droits, et le sandbox du J1 ne couvre pas les serveurs MCP : on l'approuve comme une dépendance npm ou NuGet qu'on ajoute au projet. Un serveur qui va chercher du contenu externe expose à l'injection de prompt, des instructions cachées dans ce contenu ; la documentation de Claude Code le rappelle. Les barrières du J1 tiennent : hors du mode Auto, chaque appel d'outil MCP demande votre accord (`mcp__tailspin__*` dans `permissions.allow` lève la question pour ce serveur), et un hook peut viser les outils d'un serveur.

Dans un repo, Claude a souvent un chemin plus court que votre serveur : un serveur MCP n'est pas un réflexe, c'est une réponse à un besoin.

Mesurer les résultats, pas l'activité

À éviter : mesurer l'activité, c'est-à-dire le nombre de prompts, de sessions, de lignes générées. Ça ne mesure pas la valeur, et ça pousse dans le mauvais sens : plus de lignes générées, c'est surtout plus de code à relire. C'est la loi de Goodhart, dans la formulation de Marilyn Strathern : quand une mesure devient un objectif, elle cesse d'être une bonne mesure.

À mesurer : le résultat. Des suggestions d'indicateurs, pas des seuils :

Indicateur	Ce qu'il vous dit
Le temps de relecture par PR	S'il monte, vous avez parallélisé trop vite.
La part des changements mergés du premier coup, sans aller-retour de revue	Si votre harness produit du code acceptable.
Les retours sur la spec une fois le plan lancé	Chaque retour est une question que l'interview aurait dû poser.

Le chiffre qui justifie de mesurer : en juillet 2025, 16 développeurs open source expérimentés ont mis 19 % de temps en plus sur leurs tâches avec l'IA, alors qu'ils pensaient avoir gagné 20 % (METR, qui juge aujourd'hui ces résultats dépassés ; la mise à jour de février 2026, plus favorable, est jugée peu fiable par METR lui-même). La leçon n'est pas « l'IA ralentit » : l'impression de vitesse et la vitesse mesurée peuvent aller en sens inverse. Mesurez sur vos propres tickets, avant et après.

Pour aller plus loin

Quinze lectures pour prolonger les deux jours, rangées par demi-journée. Toutes sont en anglais ; liens vérifiés le 14/10/2026. Si vous n'en lisez qu'une, prenez la première.

Une seule lecture

- Simon Willison, « Agentic Engineering Patterns », un guide en chapitres tenu à jour : comment fonctionne un agent de code, git avec un agent, les sous-agents, les tests. Il reprend presque tout le programme, avec les exemples d'un praticien : https://simonwillison.net/guides/agentic-engineering-patterns/

J1 matin : le contexte et le CLAUDE.md

- Cédric Teyton (Packmind), « Writing AI coding agent context files is easy. Keeping them accurate isn't. » (06/02/2026) : générer un CLAUDE.md prend une minute, le garder exact prend une discipline. Pourquoi il dérive avec le code, et comment l'en empêcher. Prolonge « L'entretenir » : https://packmind.com/evaluate-context-ai-coding-agent/
- Dave Farley, « We Studied 150 Developers Using AI » (vidéo, chaîne Modern Software Engineering) : une étude contrôlée sur environ 150 développeurs. L'IA les rend plus rapides sans dégrader la maintenabilité du code, et elle amplifie les bonnes pratiques comme les mauvaises. Un contrepoint aux chiffres du matin : https://www.youtube.com/watch?v=b9EbCb5A408

J1 après-midi : les skills

- Matt Pocock, « 5 Agent Skills I Use Every Day » (mis à jour le 16/03/2026) : `/grill-me`, `/to-spec`, `/to-tickets`, `/tdd` et un cinquième, chacun expliqué en quelques paragraphes. La preuve qu'un skill n'a pas besoin d'être long pour changer une journée, et le pont vers la chaîne du J2 matin : https://www.aihero.dev/5-agent-skills-i-use-every-day
- Thariq Shihipar (Anthropic), « Skills at Anthropic: Lessons from Hundreds of Skills in Active Use » (03/2026, fil sur X) : les neuf catégories de skills en usage chez Anthropic et ce qui distingue ceux qui servent : https://x.com/trq212/status/2033949937936085378
- Anthropic, le dépôt `anthropics/skills` : des skills officiels à lire comme modèles avant d'écrire le vôtre, dont `skill-creator`, qui aide à en écrire et à les tester : https://github.com/anthropics/skills

J2 matin : la spec, la supervision, les loops

- Addy Osmani, « How to Write a Good Spec for AI Agents » (19/01/2026) : comment passer d'un ticket ambigu à une spec qu'un agent peut suivre, et ce qu'elle doit couvrir. Prolonge « La spec, source de vérité » et l'Atelier 2 : https://addyo.substack.com/p/how-to-write-a-good-spec-for-ai-agents
- Simon Willison, « Fireside chat at Pragmatic Summit » (14/03/2026) : « use red-green TDD » : cinq tokens qui suffisent à un agent pour écrire le test avant le code ; et, puisque les tests ne coûtent presque plus rien, « tests are no longer even remotely optional ». À lire si vous écrivez vos tests après le code : https://simonwillison.net/2026/Mar/14/pragmatic-summit/
- Dex Horthy (HumanLayer), « A Brief History of Ralph » (06/01/2026) : d'où vient Ralph, ses variantes et ses pièges, avec le recul de six mois d'usage. Prolonge « Les loops » : https://www.humanlayer.dev/blog/brief-history-of-ralph. L'article d'origine, de Geoffrey Huntley : https://ghuntley.com/ralph/
- Anthropic, « How AI is transforming work at Anthropic » (02/12/2025) : 132 ingénieurs interrogés, 200 000 sessions Claude Code analysées. Claude enchaîne 21,2 actions sans intervention humaine, contre 9,8 six mois plus tôt ; et le « paradoxe de la supervision » : superviser Claude demande les compétences que son usage risque d'éroder. Prolonge « Superviser : in, on, outside the loop » : https://www.anthropic.com/research/how-ai-is-transforming-work-at-anthropic

- Dex Horthy (HumanLayer), « Why Software Factories Fail » (07/2026) : le récit, de l'intérieur, d'une « usine lights-off » où plus personne ne lisait le code, et pourquoi son équipe en est revenue ; ses conseils pour aller deux à trois fois plus vite, en sécurité. Prolonge « Superviser : in, on, outside the loop » : https://github.com/humanlayer/advanced-context-engineering-for-coding-agents/blob/main/wsff.md

J2 après-midi : paralléliser et mesurer

- Addy Osmani, « The 80% Problem in Agentic Coding » (28/01/2026) : l'agent livre vite les premiers 80 % ; l'effort se concentre ensuite sur les 20 % restants, et sur la « dette de compréhension » d'un code qu'on n'a pas écrit. Prolonge « Ce que coûte la parallélisation » : https://addyo.substack.com/p/the-80-problem-in-agentic-coding
- Steve Smith, « The Truth About Developer Productivity in the AI Age » (vidéo, chaîne Modern Software Engineering) : pourquoi compter les prompts, les lignes ou les PR mesure l'activité et pas les résultats, et ce qu'on peut mesurer à la place. Prolonge « Mesurer les résultats, pas l'activité » : https://www.youtube.com/watch?v=kDBeFOscZpc
- Addy Osmani, « Software Factories, Light and Dark » (20/07/2026) : la vérification, pas la génération, est la vraie contrainte d'une usine logicielle ; on ne donne à une boucle que l'autonomie qu'on peut vérifier. Prolonge « Le coût humain » : https://addyosmani.com/blog/software-factories/
- IndyDevDan, « Agent Threads » (vidéo, 12/01/2026) : quatre leviers pour progresser avec les agents : plus de threads en parallèle, des threads plus longs, plus épais (des sous-agents), moins de checkpoints humains. À regarder avec trois réserves : ce sont des leviers, pas des indicateurs, et sa mesure (le nombre d'appels d'outils de vos agents) compte l'activité, pas le résultat ; plus de threads, seulement si votre relecture suit ; plus épais, sans cascade, et moins de checkpoints, seulement là où le harness vérifie. Prolonge « Mesurer les résultats, pas l'activité » : https://www.youtube.com/watch?v=-WBHNFAB0OE

Sources des deux jours

Chiffres cités

- Stack Overflow, Developer Survey 2026 (06/10/2026, environ 31 000 réponses) : https://survey.stackoverflow.co/2026
- JetBrains, State of Developer Ecosystem 2025 (21/10/2025, 24 534 développeurs) : https://blog.jetbrains.com/research/2025/10/state-of-developer-ecosystem-2025/
- JetBrains, enquête 2026, résultats partiels (collectés de mai à juillet 2026) : 90 % des développeurs professionnels utilisent des agents de code au moins chaque semaine, 68 % chaque jour.
- METR, étude sur 16 développeurs open source expérimentés (10/07/2025) : https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/
- METR, mise à jour (24/02/2026) : https://metr.org/blog/2026-02-24-uplift-update/
- Anthropic, « How we built our multi-agent research system » (13/06/2025) : environ 4 fois les tokens d'un chat pour un agent, 15 fois pour un système multi-agents.
- Gloaguen et al. (ETH Zurich, LogicStar), « Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? », v3 (29/09/2026) : https://arxiv.org/abs/2602.11988
- Bedard et al. (BCG), « When Using AI Leads to "Brain Fry" », Harvard Business Review (05/03/2026) : https://hbr.org/2026/03/when-using-ai-leads-to-brain-fry
- Simon Willison, « Highlights from my conversation about agentic engineering on Lenny's Podcast » (02/04/2026) : https://simonwillison.net/2026/Apr/2/lennys-podcast/
- Faros AI, « AI Engineering Report 2026: The Acceleration Whiplash » (04/2026 ; 22 000 développeurs, 4 000 équipes, deux ans ; chaque équipe comparée à elle-même) : durée médiane de l'étape de revue des PR +441,5 %, incidents par PR +242,7 % : https://www.faros.ai/research/ai-acceleration-whiplash
- Anthropic, tarifs de l'API (10/2026) : Opus 5.5 à 4 $ / 20 $ par million de tokens en entrée / sortie, Sonnet 5.5 à 2 $ / 10 $: https://platform.claude.com/docs/en/about-claude/pricing

Contexte et context engineering

- Anthropic, « Effective context engineering for AI agents » (29/09/2025) : https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Andrej Karpathy (juin 2025), paraphrase : le context engineering, c'est l'art de remplir la fenêtre de contexte avec exactement l'information utile à l'étape suivante.
- Chroma, « Context Rot » (14/07/2025) : https://www.trychroma.com/research/context-rot
- Anthropic, « The new rules of context engineering for Claude 5 generation models » (Thariq Shihipar, 24/07/2026) : https://claude.com/blog/the-new-rules-of-context-engineering-for-claude-5-generation-models
- Anthropic, « The AI-native SDLC playbook » (21/08/2026), figure 1, « The bottleneck moved » (le goulot s'est déplacé) : https://claude.com/resources/articles/the-ai-native-sdlc-playbook
- Liu et al., « Lost in the Middle » (2023-2024) : https://arxiv.org/abs/2307.03172
- Veseli et al. (08/2025), sur la portée de cet effet : https://arxiv.org/abs/2508.07479
- Drew Breunig, « How Long Contexts Fail » (22/06/2025) : https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html
- Drew Breunig, « How to Fix Your Context » (26/06/2025) : https://www.dbreunig.com/2025/06/26/how-to-fix-your-context.html

Spec, workflows et méthodes

- Birgitta Böckeler, sur le spec-driven development (spec-first, spec-anchored, spec-as-source ; Spec-Kit, Kiro, Tessl), martinfowler.com (15/10/2025).
- Matt Pocock, README de ses skills : les méthodes « possèdent le processus », ses skills se composent : https://github.com/mattpocock/skills
- Les méthodes clé en main, d'après leurs dépôts et leurs sites au 14/10/2026 (Kiro, qui n'est pas open source : kiro.dev, dont le billet « Faster, smarter specs » du 12/05/2026) : Spec-Kit (GitHub, v1.2.0) ; Kiro (AWS, CLI 2.31) ; BMAD (v6.12.2 stable, v7 en préversion) ; GSD Core (open-gsd/gsd-core, v1.16 ; le dépôt d'origine de GSD est

archivé depuis 06/2026) ; Superpowers (v7.0.0 sur son dépôt, 6.4.1 épinglée dans la marketplace officielle).
- Les critiques citées : Marmelab, « Spec-Driven Development: The Waterfall Strikes Back », sur le spec-driven en général (12/11/2025) : https://marmelab.com/blog/2025/11/12/spec-driven-development-waterfall-strikes-back.html ; Hicron, sur BMAD et GSD (27/05/2026) : https://hicronsoftware.com/blog/bmad-gsd-ai-assisted-software-dev-work/ ; MCP.Directory, sur Superpowers (12/06/2026) : https://mcp.directory/blog/superpowers-skill-worth-it-2026.
- Manfred Riem, mainteneur principal de Spec-Kit, cité par Visual Studio Magazine (12/05/2026) : le plan mode peut suffire pour du travail ponctuel ; la fiabilité baisse après une compaction, une limite des LLM selon lui : https://visualstudiomagazine.com/articles/2026/05/12/github-spec-kit-takes-off-as-antidote-to-piecemeal-vibe-coding.aspx

Harness et supervision

- Kief Morris, « Humans and Agents in Software Engineering Loops », martinfowler.com (04/03/2026) : https://martinfowler.com/articles/exploring-gen-ai/humans-and-agents.html
- Florian Bruniaux, « Agent Harness Map » (2026), les quatre couches de l'écosystème : https://cc.bruniaux.com/cheatsheets/c14-agent-harness-map/
- Addy Osmani, « Software Factories, Light and Dark » (20/07/2026) : https://addyosmani.com/blog/software-factories/
- Dex Horthy, « Why Software Factories Fail » (07/2026) : https://github.com/humanlayer/advanced-context-engineering-for-coding-agents/blob/main/wsff.md

Loops

- Anthropic, « Loop engineering: Getting started with loops », claude.com/blog (30/06/2026) : les 4 types de loop.
- Anthropic, « Effective harnesses for long-running agents » (26/11/2025) : l'état persistant, un fichier de progression et des commits réguliers.
- Simon Willison, « Designing agentic loops » (30/09/2025) : le budget, le sandbox. Le run pilote vient du billet d'Anthropic sur les loops (« Pilot before a large run »).
- Geoffrey Huntley, Ralph (07/2025) : une boucle bash qui relance le même prompt, avec les tests et le typage comme contre-pression.
- Le plugin `ralph-loop` (v1.0.0) de la marketplace officielle (lien plus bas).

Parallélisation

- Anthropic, « How we built our multi-agent research system » (13/06/2025) : l'architecture orchestrateur-workers ; les quatre points du brief d'un sous-agent ; « most coding tasks involve fewer truly parallelizable tasks than research ».
- Matt Pocock, documentation de `/to-tickets` (le découpage par couche : 26 tickets, environ 20 passages par ticket fermé) et de `/implement` (plusieurs sessions dans un même checkout : « worse than unsupported », d'après un retour d'utilisateur qu'il cite) : https://github.com/mattpocock/skills
- Matt Pocock, issue #573 de `mattpocock/skills`, « code-review sub-agents recursively invoke /code-review and fan out » (14/07/2026, fermée sans correctif le 06/10/2026) : https://github.com/mattpocock/skills/issues/573
- Dex Horthy, « No Vibes Allowed: Solving Hard Problems in Complex Codebases » (AI Engineer Code Summit, New York, novembre 2025 ; vidéo en ligne le 02/12/2025), à 6:35, sur les rôles : https://www.youtube.com/watch?v=rmvDxxNubIg
- IndyDevDan, « Agent Threads » (12/01/2026) : https://www.youtube.com/watch?v=-WBHNFAB0OE

MCP

- Model Context Protocol, « Build an MCP server », le tutoriel officiel (en TypeScript, C#, Java, Python, Kotlin, Go, Rust et Ruby) : les outils, les ressources et les prompts d'un serveur ; jamais rien sur stdout en stdio : https://modelcontextprotocol.io/docs/develop/build-server
- Le SDK officiel en TypeScript, en v2 depuis le 27/07/2026 (`@modelcontextprotocol/server` 2.3.1 au 15/10/2026 ; la v1, `@modelcontextprotocol/sdk`, ne reçoit plus que des correctifs) : https://github.com/modelcontextprotocol/typescript-sdk

- Atlassian, le serveur MCP de Jira, Confluence et Bitbucket (Rovo MCP Server : distant, hébergé, pour Atlassian Cloud) : https://github.com/atlassian/atlassian-mcp-server
- Microsoft, le serveur MCP d'Azure DevOps (officiel, en préversion, local ou distant) : https://github.com/microsoft/azure-devops-mcp

Exemples et outils

- Bénéfriches, le projet de démo du J1 (licence MIT ; commit `bec2057` du 12/10/2026) : https://github.com/incubateur-ademe/benefriches
- Tailspin Toys, le projet d'entraînement du J2 : le fork `denneulin/tailspin-toys`, préparé pour Claude Code (un CLAUDE.md, quatre règles, les permissions des tests, neuf tickets dans `backlog/`, des branches de secours `secours/*`), de `github-samples/tailspin-toys` (licence MIT) : https://github.com/denneulin/tailspin-toys
- Les skills de Matt Pocock (`/grill-me`, `/grill-with-docs`, `/to-spec`, `/to-tickets`, `/implement`, `/tdd`, `/code-review`, `/setup-matt-pocock-skills`), licence MIT ; plugin `mattpocock-skills`, affiché 1.2.3 sur la marketplace officielle, 1.3.1 sur celle de Matt Pocock ; `setup-pre-commit` est hors du plugin, dans `skills/misc/` : https://github.com/mattpocock/skills
- La marketplace officielle de plugins : https://github.com/anthropics/claude-plugins-official

Documentation Claude Code (en français)

- Bonnes pratiques : https://code.claude.com/docs/fr/best-practices
- La fenêtre de contexte : https://code.claude.com/docs/fr/context-window
- Commandes : https://code.claude.com/docs/fr/commands
- CLAUDE.md, règles et mémoire : https://code.claude.com/docs/fr/memory
- Hooks, guide : https://code.claude.com/docs/fr/hooks-guide
- Hooks, référence : https://code.claude.com/docs/fr/hooks
- Permissions : https://code.claude.com/docs/fr/permissions
- Sandbox : https://code.claude.com/docs/fr/sandboxing
- Skills : https://code.claude.com/docs/fr/skills
- Plugins d'intelligence de code (LSP) : https://code.claude.com/docs/fr/plugins/code-intelligence
- MCP : https://code.claude.com/docs/fr/mcp

Documentation Claude Code, pages du J2

Vérifiées le 13/10/2026 sur la documentation officielle en anglais (code.claude.com/docs/en) :

- « Best practices » : l'interview puis la spec dans un fichier, les quatre phases, les quatre façons de rendre la vérification obligatoire, le pattern Writer / Reviewer (en français : Bonnes pratiques, ci-dessus).
- « goal » : `/goal`, l'évaluateur, l'écriture d'une condition.
- « worktrees » : `claude --worktree`, `worktree.baseRef`, `.worktreeinclude`.
- « sub-agents » : les sous-agents intégrés et personnalisés, leur cycle de vie.
- « Run agents in parallel » : les trois questions pour choisir entre sous-agents, sessions, agent teams et scripts.
- « costs » et « model-config » (vérifiées le 14/10/2026) : choisir le modèle, `--model` et `/model`, le modèle d'un sous-agent ; et, page « sub-agents », l'imbrication et `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`.
- « Scheduled tasks » (vérifiée le 14/10/2026) : `/loop`, les tâches planifiées du desktop, les Routines.
- « MCP » (vérifiée le 15/10/2026, avec Claude Code 2.1.301) : `claude mcp add`, les portées et `.mcp.json`, l'approbation, les noms `mcp__<serveur>__<outil>`, la taille des sorties, l'injection de prompt ; et, page « sub-agents », le champ `mcpServers`.